

NETWORK PROTOCOLS — VLAN, EAP, WI-FI AND SUPPLICANT

27.1 VLAN Configuration Protocol

27.1.1 EFI_VLAN_CONFIG_PROTOCOL

Summary

This protocol is to provide manageability interface for VLAN configuration.

GUID

```
#define EFI_VLAN_CONFIG_PROTOCOL_GUID \
    {0x9e23d768, 0xd2f3, 0x4366, \
     {0x9f, 0xc3, 0x3a, 0x7a, 0xba, 0x86, 0x43, 0x74}}
```

Protocol Interface Structure

```
typedef struct _EFI_VLAN_CONFIG_PROTOCOL {
    EFI_VLAN_CONFIG_SET          Set;
    EFI_VLAN_CONFIG_FIND         Find;
    EFI_VLAN_CONFIG_REMOVE       Remove;
} EFI_VLAN_CONFIG_PROTOCOL;
```

Parameters

Set

Create new VLAN device or modify configuration parameter of an already-configured VLAN

Find

Find configuration information for specified VLAN or all configured VLANs.

Remove

Remove a VLAN device.

Description

This protocol is to provide manageability interface for VLAN setting. The intended VLAN tagging implementation is IEEE802.1Q.

27.1.2 EFI_VLAN_CONFIG_PROTOCOL.Set()

Summary

Create a VLAN device or modify the configuration parameter of an already-configured VLAN

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_VLAN_CONFIG_SET) (
    IN EFI_VLAN_CONFIG_PROTOCOL    *This,
    IN UINT16                      VlanId,
    IN UINT8                      Priority
);
```

Parameters

This

Pointer to `EFI_VLAN_CONFIG_PROTOCOL` instance.

VlanId

A unique identifier (1-4094) of the VLAN which is being created or modified, or zero (0).

Priority

3 bit priority in VLAN header. Priority 0 is default value. If `VlanId` is zero (0), `Priority` is ignored.

Description

The *Set()* function is used to create a new VLAN device or change the VLAN configuration parameters. If the *VlanId* hasn't been configured in the physical Ethernet device, a new VLAN device will be created. If a VLAN with this *VlanId* is already configured, then related configuration will be updated as the input parameters.

If *VlanId* is zero, the VLAN device will send and receive untagged frames. Otherwise, the VLAN device will send and receive VLAN-tagged frames containing the *VlanId*.

If *VlanId* is out of scope of (0-4094), *EFI_INVALID_PARAMETER* is returned

If *Priority* is out of the scope of (0-7), then *EFI_INVALID_PARAMETER* is returned.

If there is not enough system memory to perform the registration, then *EFI_OUT_OF_RESOURCES* is returned.

Status Codes Returned

EFI_SUCCESS	The VLAN is successfully configured
EFI_INVALID_PARAMETER	One or more of following conditions is TRUE • This is NULL • VlanId is an invalid VLAN Identifier • Priority is invalid
EFI_OUT_OF_RESOURCES	There is not enough system memory to perform the registration.

27.1.3 EFI_VLAN_CONFIG_PROTOCOL.Find()

Summary

Find configuration information for specified VLAN or all configured VLANs.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_VLAN_CONFIG_FIND) (
    IN EFI_VLAN_CONFIG_PROTOCOL    *This,
    IN UINT16                      *VlanId, OPTIONAL
    OUT UINT16                     *NumberOfVlan,
    OUT EFI_VLAN_FIND_DATA         **Entries
);
```

Parameters

This

Pointer to *EFI_VLAN_CONFIG_PROTOCOL* instance.

VlanId

Pointer to VLAN identifier. Set to *NULL* to find all configured VLANs

NumberOfVlan

The number of VLANs which is found by the specified criteria

Entries

The buffer which receive the VLAN configuration. Type *EFI_VLAN_FIND_DATA* is defined below.

Description

The *Find()* function is used to find the configuration information for matching VLAN and allocate a buffer into which those entries are copied.

Related Definitions

```
/**
 *
 */
// EFI_VLAN_FIND_DATA
/**
 *
 */
typedef struct {
    UINT16      VlanId;
    UINT8       Priority;
} EFI_VLAN_FIND_DATA;
```

VlanId

Vlan Identifier

Priority

Priority of this VLAN

Status Codes Returned

EFI_SUCCESS	The VLAN is successfully found
-------------	--------------------------------

continues on next page

Table 27.2 – continued from previous page

EFI_INVALID_PARAMETER	One or more of following conditions is <i>TRUE</i> • <i>This</i> is <i>NULL</i> • Specified <i>VlanId</i> is invalid
EFI_NOT_FOUND	No matching VLAN is found

27.1.4 EFI_VLAN_CONFIG_PROTOCOL.Remove()

Summary

Remove the configured VLAN device

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_VLAN_CONFIG_REMOVE) (
    IN EFI_VLAN_CONFIG_PROTOCOL      *This,
    IN UINT16                        VlanId
);
```

Parameters

This

Pointer to EFI_VLAN_CONFIG_PROTOCOL instance.

VlanId

Identifier (0-4094) of the VLAN to be removed.

Description

The *Remove()* function is used to remove the specified VLAN device. If the *VlanId* is out of the scope of (0-4094), *EFI_INVALID_PARAMETER* is returned. If specified VLAN hasn't been previously configured, *EFI_NOT_FOUND* is returned.

Status Codes Returned

EFI_SUCCESS	The VLAN is successfully removed
EFI_INVALID_PARAMETER	One or more of following conditions is <i>TRUE</i> • <i>This</i> is <i>NULL</i> • <i>VlanId</i> is an invalid parameter.
EFI_NOT_FOUND	The to-be-removed VLAN does not exist

27.2 EAP Protocol

This section defines the EAP protocol. This protocol is designed to make the EAP framework configurable and extensible. It is intended for the supplicant side.

27.2.1 EFI_EAP_PROTOCOL

Summary

This protocol is used to abstract the ability to configure and extend the EAP framework.

GUID

```
#define EFI_EAP_PROTOCOL_GUID \
{ 0x5d9f96db, 0xe731, 0x4caa, \
  {0xa0, 0x0d, 0x72, 0xe1, 0x87, 0xcd, 0x77, 0x62 } }
```

Protocol Interface Structure

```
typedef struct _EFI_EAP_PROTOCOL {
    EFI_EAP_SET_DESIRED_AUTHENTICATION_METHOD    SetDesiredAuthMethod;
    EFI_EAP_REGISTER_AUTHENTICATION_METHOD       RegisterAuthMethod;
} EFI_EAP_PROTOCOL;
```

Parameters

SetDesiredAuthMethod

Set the desired EAP authentication method for the Port. See the *SetDesiredAuthMethod()* function description.

RegisterAuthMethod

Register an EAP authentication method. See the *RegisterAuthMethod()* function description.

Description

EFI_EAP_PROTOCOL is used to configure the desired EAP authentication method for the EAP framework and extend the EAP framework by registering new EAP authentication method on a Port. The EAP framework is built on a per-Port basis. Herein, a Port means a NIC. For the details of EAP protocol, please refer to RFC 2284.

Related Definitions

```
//
// Type for the identification number assigned to the Port by the
// System in which the Port resides.
//
typedef VOID * EFI_PORT_HANDLE;
```

27.2.2 EFI_EAP.SetDesiredAuthMethod()

Summary

Set the desired EAP authentication method for the Port.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_SET_DESIRED_AUTHENTICATION_METHOD) (
    IN struct _EFI_EAP_PROTOCOL      *This,
    IN UINT8                        EapAuthType
);
```

Parameters

This

A pointer to the *EFI_EAP_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_PROTOCOL* is defined in Section 1.1.

EapAuthType

The type of the desired EAP authentication method for the Port. It should be the type value defined by RFC. See RFC 2284 for details. Current valid values are defined in “Related Definitions”.

Related Definitions

```
//
// EAP Authentication Method Type (RFC 3748)
//
#define EFI_EAP_TYPE_TLS 13 /* REQUIRED - RFC 5216 */
```

Description

The *SetDesiredAuthMethod()* function sets the desired EAP authentication method indicated by *EapAuthType* for the Port.

If *EapAuthType* is an invalid EAP authentication type, then *EFI_INVALID_PARAMETER* is returned.

If the EAP authentication method of *EapAuthType* is unsupported, then it will return *EFI_UNSUPPORTED*.

Status Codes Returned

EFI_SUCCESS	The desired EAP authentication method is set successfully.
EFI_INVALID_PARAMETER	EapAuthType is an invalid EAP authentication type.
EFI_UNSUPPORTED	The EAP authentication method of EapAuthType is unsupported by the Port.

27.2.3 EFI_EAP.RegisterAuthMethod()

Summary

Register an EAP authentication method.

Prototype

```
typedef
EFI_STATUS
```

(continues on next page)

(continued from previous page)

```
(EFIAPI *EFI_EAP_REGISTER_AUTHENTICATION_METHOD) (
    IN struct _EFI_EAP_PROTOCOL      *This,
    IN UINT8                        EapAuthType,
    IN EFI_EAP_BUILD_RESPONSE_PACKET Handler
);
```

Parameters

This

A pointer to the *EFI_EAP_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_PROTOCOL* is defined in Section 1.1.

EapAuthType

The type of the EAP authentication method to register. It should be the type value defined by RFC. See RFC 2284 for details. Current valid values are defined in the *SetDesiredAuthMethod()* function description.

Handler

The handler of the EAP authentication method to register. Type *EFI_EAP_BUILD_RESPONSE_PACKET* is defined in “Related Definitions”.

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_BUILD_RESPONSE_PACKET) (
    IN EFI_PORT_HANDLE      PortNumber
    IN UINT8                *RequestBuffer,
    IN UINTN                RequestSize,
    IN UINT8                *Buffer,
    IN OUT UINTN            *BufferSize
);
```

Routine Description:

Build EAP response packet in response to the EAP request packet specified by (RequestBuffer, RequestSize).

Arguments

PortNumber — Specified the Port where the EAP request packet comes.
 RequestBuffer — Pointer to the most recently received EAP-Request packet.
 RequestSize — Packet size in bytes for the most recently received EAP-Request packet.
 Buffer — Pointer to the buffer to hold the built packet.
 BufferSize — Pointer to the buffer size in bytes. On input, it is the buffer size provided by the caller. On output, it is the buffer size in fact needed to contain the packet.

Returns:

EFI_SUCCESS — The required EAP response packet is built successfully. Others — Failures are encountered during the packet building process.

Description

The *RegisterAuthMethod()* function registers the user provided EAP authentication method, the type of which is *EapAuthType* and the handler of which is *Handler*.

If *EapAuthType* is an invalid EAP authentication type, then *EFI_INVALID_PARAMETER* is returned.

If there is not enough system memory to perform the registration, then *EFI_OUT_OF_RESOURCES* is returned.

Status Codes Returned

EFI_SUCCESS	The EAP authentication method of <i>EapAuthType</i> is registered successfully.
EFI_INVALID_PARAMETER	<i>EapAuthType</i> is an invalid EAP authentication type.
EFI_OUT_OF_RESOURCES	There is not enough system memory to perform the registration.

27.2.4 EAPManagement Protocol

This section defines the EAP management protocol. This protocol is designed to provide ease of management and ease of test for EAPOL state machine. It is intended for the supplicant side. It conforms to IEEE 802.1x specification.

27.2.5 EFI_EAP_MANAGEMENT_PROTOCOL**Summary**

This protocol provides the ability to configure and control EAPOL state machine, and retrieve the status and the statistics information of EAPOL state machine.

GUID

```
#define EFI_EAP_MANAGEMENT_PROTOCOL_GUID \
{ 0xbb62e663, 0x625d, 0x40b2, \
  { 0xa0, 0x88, 0xbb, 0xe8, 0x36, 0x23, 0xa2, 0x45 } }
```

Protocol Interface Structure

```
typedef struct _EFI_EAP_MANAGEMENT_PROTOCOL {
    EFI_EAP_GET_SYSTEM_CONFIGURATION      GetSystemConfiguration;
    EFI_EAP_SET_SYSTEM_CONFIGURATION      SetSystemConfiguration;
    EFI_EAP_INITIALIZE_PORT                InitializePort;
    EFI_EAP_USER_LOGON                     UserLogon;
    EFI_EAP_USER_LOGOFF                    UserLogoff;
    EFI_EAP_GET_SUPPLICANT_STATUS           GetSupplicantStatus;
    EFI_EAP_SET_SUPPLICANT_CONFIGURATION   SetSupplicantConfiguration;
    EFI_EAP_GET_SUPPLICANT_STATISTICS       GetSupplicantStatistics;
} EFI_EAP_MANAGEMENT_PROTOCOL;
```

Parameters**GetSystemConfiguration**

Read the system configuration information associated with the Port. See the *GetSystemConfiguration()* function description.

SetSystemConfiguration

Set the system configuration information associated with the Port. See the *SetSystemConfiguration()* function description.

InitializePort

Cause the EAPOL state machines for the Port to be initialized. See the *InitializePort()* function description.

UserLogon

Notify the EAPOL state machines for the Port that the user of the System has logged on. See the *UserLogon()* function description.

UserLogoff

Notify the EAPOL state machines for the Port that the user of the System has logged off. See the *UserLogoff()* function description.

GetSupplicantStatus

Read the status of the Supplicant PAE state machine for the Port, including the current state and the configuration of the operational parameters. See the *GetSupplicantStatus()* function description.

SetSupplicantConfiguration

Set the configuration of the operational parameter of the Supplicant PAE state machine for the Port. See the *SetSupplicantConfiguration()* function description.

GetSupplicantStatistics

Read the statistical information regarding the operation of the Supplicant associated with the Port. See the *GetSupplicantStatistics()* function description.

Description

The *EFI_EAP_MANAGEMENT* protocol is used to control, configure and monitor EAPOL state machine on a Port. EAPOL state machine is built on a per-Port basis. Herein, a Port means a NIC. For the details of EAPOL, please refer to IEEE 802.1x specification.

27.2.6 EFI_EAP_MANAGEMENT.GetSystemConfiguration()

Summary

Read the system configuration information associated with the Port.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_GET_SYSTEM_CONFIGURATION) (
    IN struct _EFI_EAP_MANAGEMENT_PROTOCOL *This,
    OUT BOOLEAN                            *SystemAuthControl,
    OUT EFI_EAPOL_PORT_INFO                *PortInfo OPTIONAL
);
```

Parameters

This

A pointer to the *EFI_EAP_MANAGEMENT_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_MANAGEMENT_PROTOCOL* is defined in *EAPManagement Protocol*.

SystemAuthControl

Returns the value of the SystemAuthControl parameter of the System. *TRUE* means Enabled. *FALSE* means Disabled.

PortInfo

Returns *EFI_EAPOL_PORT_INFO* structure to describe the Port's information. This parameter can be NULL to ignore reading the Port's information. Type *EFI_EAPOL_PORT_INFO* is defined in "Related Definitions".

Related Definitions

```
//
// PAE Capabilities
//
#define PAE_SUPPORT_AUTHENTICATOR 0x01
```

(continues on next page)

(continued from previous page)

```
#define PAE_SUPPORT_SUPPLICANT    0x02

typedef struct _EFI_EAPOL_PORT_INFO {
    EFI_PORT_HANDLE      PortNumber;
    UINT8                ProtocolVersion;
    UINT8                PaeCapabilities;
} EFI_EAPOL_PORT_INFO;
```

PortNumber

The identification number assigned to the Port by the System in which the Port resides.

ProtocolVersion

The protocol version number of the EAPOL implementation supported by the Port.

PaeCapabilities

The capabilities of the PAE associated with the Port. This field indicates whether Authenticator functionality, Supplicant functionality, both, or neither, is supported by the Port's PAE.

Description

The *GetSystemConfiguration()* function reads the system configuration information associated with the Port, including the value of the SystemAuthControl parameter of the System is returned in *SystemAuthControl* and the Port's information is returned in the buffer pointed to by PortInfo. The Port's information is optional. If *PortInfo* is *NULL*, then reading the Port's information is ignored.

If *SystemAuthControl* is *NULL*, then *EFI_INVALID_PARAMETER* is returned.

Status Codes Returned

EFI_SUCCESS	The system configuration information of the Port is read successfully.
EFI_INVALID_PARAMETER	SystemAuthControl is NULL.

27.2.7 EFI_EAP_MANAGEMENT.SetSystemConfiguration()**Summary**

Set the system configuration information associated with the Port.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_SET_SYSTEM_CONFIGURATION) (
    IN struct _EFI_EAP_MANAGEMENT_PROTOCOL    *This,
    IN BOOLEAN                                SystemAuthControl
);
```

Parameters**This**

A pointer to the *EFI_EAP_MANAGEMENT_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_MANAGEMENT_PROTOCOL* is defined in *EAPManagement Protocol*.

SystemAuthControl

The desired value of the SystemAuthControl parameter of the System. *TRUE* means Enabled. *FALSE* means Disabled.

Description

The *SetSystemConfiguration()* function sets the value of the SystemAuthControl parameter of the System to *SystemAuthControl*.

Status Codes Returned

EFI_SUCCESS	The system configuration information of the Port is set successfully.
-------------	---

27.2.8 EFI_EAP_MANAGEMENT.InitializePort()

Summary

Cause the EAPOL state machines for the Port to be initialized.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_INITIALIZE_PORT) (
    IN struct _EFI_EAP_MANAGEMENT_PROTOCOL *This
);
```

Parameters

This

A pointer to the *EFI_EAP_MANAGEMENT_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_MANAGEMENT_PROTOCOL* is defined in [EAPManagement Protocol](#).

Description

The InitializePort() function causes the EAPOL state machines for the Port.

Status Codes Returned

EFI_SUCCESS	The Port is initialized successfully.
-------------	---------------------------------------

27.2.9 EFI_EAP_MANAGEMENT.UserLogon()

Summary

Notify the EAPOL state machines for the Port that the user of the System has logged on.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_USER_LOGON) (
    IN struct _EFI_EAP_MANAGEMENT_PROTOCOL *This,
);
```

Parameters

This

A pointer to the *EFI_EAP_MANAGEMENT_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_MANAGEMENT_PROTOCOL* is defined in [EAPManagement Protocol](#).

Description

The UserLogon() function notifies the EAPOL state machines for the Port.

Status Codes Returned

EFI_SUCCESS	The Port is notified successfully.
-------------	------------------------------------

27.2.10 EFI_EAP_MANAGEMENT.UserLogoff()**Summary**

Notify the EAPOL state machines for the Port that the user of the System has logged off.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_USER_LOGOFF) (
    IN struct _EFI_EAP_MANAGEMENT_PROTOCOL *This,
);
```

Parameters**This**

A pointer to the *EFI_EAP_MANAGEMENT_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_MANAGEMENT_PROTOCOL* is defined in *EAPManagement Protocol*.

Description

The UserLogoff() function notifies the EAPOL state machines for the Port.

Status Codes Returned

EFI_SUCCESS	The Port is notified successfully.
-------------	------------------------------------

27.2.11 EFI_EAP_MANAGEMENT.GetSupplicantStatus()**Summary**

Read the status of the Supplicant PAE state machine for the Port, including the current state and the configuration of the operational parameters.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_GET_SUPPLICANT_STATUS) (
    IN struct _EFI_EAP_MANAGEMENT_PROTOCOL *This,
    OUT EFI_EAPOL_SUPPLICANT_PAE_STATE *CurrentState,
    IN OUT EFI_EAPOL_SUPPLICANT_PAE_CONFIGURATION *Configuration OPTIONAL
);
```

Parameters

This

A pointer to the *EFI_EAP_MANAGEMENT_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_MANAGEMENT_PROTOCOL* is defined in *EAPManagement Protocol*.

CurrentState

Returns the current state of the Supplicant PAE state machine for the Port. Type *EFI_EAPOL_SUPPLICANT_PAE_STATE* is defined in “Related Definitions”.

Configuration

Returns the configuration of the operational parameters of the Supplicant PAE state machine for the Port as required. This parameter can be *NULL* to ignore reading the configuration. On input, *Configuration*. *ValidFieldMask* specifies the operational parameters to be read. On output, *Configuration* returns the configuration of the required operational parameters. Type *EFI_EAPOL_SUPPLICANT_PAE_CONFIGURATION* is defined in “Related Definitions”.

Related Definitions

```
//
// Supplicant PAE state machine (IEEE Std 802.1X Section 8.5.10)
//
typedef enum _EFI_EAPOL_SUPPLICANT_PAE_STATE {
    Logoff,
    Disconnected,
    Connecting,
    Acquired,
    Authenticating,
    Held,
    Authenticated,
    MaxSupplicantPaeState
} EFI_EAPOL_SUPPLICANT_PAE_STATE;

//
// Definitions for ValidFieldMask
//
#define AUTH_PERIOD_FIELD_VALID    0x01
#define HELD_PERIOD_FIELD_VALID    0x02
#define START_PERIOD_FIELD_VALID    0x04
#define MAX_START_FIELD_VALID    0x08

typedef struct _EFI_EAPOL_SUPPLICANT_PAE_CONFIGURATION {
    UINT8            ValidFieldMask;
    UINTN            AuthPeriod;
    UINTN            HeldPeriod;
    UINTN            StartPeriod;
    UINTN            MaxStart;
} EFI_EAPOL_SUPPLICANT_PAE_CONFIGURATION;
```

ValidFieldMask

Indicates which of the following fields are valid.

AuthPeriod

The initial value for the authWhile timer. Its default value is 30 s.

HeldPeriod

The initial value for the heldWhile timer. Its default value is 60 s.

StartPeriod

The initial value for the startWhen timer. Its default value is 30 s.

MaxStart

The maximum number of successive EAPOL-Start messages will be sent before the Supplicant assumes that there is no Authenticator present. Its default value is 3.

Description

The *GetSupplicantStatus()* function reads the status of the Supplicant PAE state machine for the Port, including the current state *CurrentState* and the configuration of the operational parameters *Configuration*. The configuration of the operational parameters is optional. If *Configuration* is *NULL*, then reading the configuration is ignored. The operational parameters in *Configuration* to be read can also be specified by *Configuration.ValidFieldMask*.

If *CurrentState* is *NULL*, then *EFI_INVALID_PARAMETER* is returned.

Status Codes Returned

EFI_SUCCESS	The status of the Supplicant PAE state machine for the Port is read successfully.
EFI_INVALID_PARAMETER	<i>CurrentState</i> is <i>NULL</i> .

27.2.12 EFI_EAP_MANAGEMENT.SetSupplicantConfiguration()

Summary

Set the configuration of the operational parameter of the Supplicant PAE state machine for the Port.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_SET_SUPPLICANT_CONFIGURATION) (
    IN struct _EFI_EAP_MANAGEMENT_PROTOCOL      *This,
    IN EFI_EAPOL_SUPPLICANT_PAE_CONFIGURATION *Configuration
);
```

Parameters

This

A pointer to the *EFI_EAP_MANAGEMENT_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_MANAGEMENT_PROTOCOL* is defined in *EAPManagement Protocol*.

Configuration

The desired configuration of the operational parameters of the Supplicant PAE state machine for the Port as required. Type *EFI_EAPOL_SUPPLICANT_PAE_CONFIGURATION* is defined in the *GetSupplicantStatus()* function description.

Description

The SetSupplicantConfiguration() function sets the configuration of the operational parameter of the Supplicant PAE state machine for the Port to Configuration. The operational parameters in Configuration to be set can be specified by Configuration.ValidFieldMask.

If Configuration is *NULL*, then *EFI_INVALID_PARAMETER* is returned.

Status Codes Returned

EFI_SUCCESS	The configuration of the operational parameter of the Supplicant PAE state machine for the Port is set successfully.
EFI_INVALID_PARAMETER	<i>Configuration is NULL.</i>

27.2.13 EFI_EAP_MANAGEMENT.GetSupplicantStatistics()

Summary

Read the statistical information regarding the operation of the Supplicant associated with the Port.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_GET_SUPPLICANT_STATISTICS) (
    IN struct _EFI_EAP_MANAGEMENT_PROTOCOL      *This,
    OUT EFI_EAPOL_SUPPLICANT_PAE_STATISTICS    *Statistics
);
```

Parameters

This

A pointer to the *EFI_EAP_MANAGEMENT_PROTOCOL* instance that indicates the calling context. Type *EFI_EAP_MANAGEMENT_PROTOCOL* is defined in [EAPManagement Protocol](#).

Statistics

Returns the statistical information regarding the operation of the Supplicant for the Port. Type *EFI_EAPOL_SUPPLICANT_PAE_STATISTICS* is defined in “Related Definitions”.

Related Definitions

```
//
// Supplicant Statistics (IEEE Std 802.1X Section 9.5.2)
//
typedef struct _EFI_EAPOL_SUPPLICANT_PAE_STATISTICS {
    UINTN  EapolFramesReceived;
    UINTN  EapolFramesTransmitted;
    UINTN  EapolStartFramesTransmitted;
    UINTN  EapolLogoffFramesTransmitted;
    UINTN  EapRespIdFramesTransmitted;
    UINTN  EapResponseFramesTransmitted;
    UINTN  EapReqIdFramesReceived;
    UINTN  EapRequestFramesReceived;
    UINTN  InvalidEapolFramesReceived;
    UINTN  EapLengthErrorFramesReceived;
    UINTN  LastEapolFrameVersion;
    UINTN  LastEapolFrameSource;
} EFI_EAPOL_SUPPLICANT_PAE_STATISTICS;
```

EapolFramesReceived

The number of EAPOL frames of any type that have been received by this Supplicant.

EapolFramesTransmitted

The number of EAPOL frames of any type that have been transmitted by this Supplicant.

EapolStartFramesTransmitted

The number of EAPOL Start frames that have been transmitted by this Supplicant.

EapolLogoffFramesTransmitted

The number of EAPOL Logoff frames that have been transmitted by this Supplicant.

EapRespIdFramesTransmitted

The number of EAP Resp/Id frames that have been transmitted by this Supplicant.

EapResponseFramesTransmitted

The number of valid EAP Response frames (other than Resp/Id frames) that have been transmitted by this Supplicant.

EapReqIdFramesReceived

The number of EAP Req/Id frames that have been received by this Supplicant.

EapRequestFramesReceived

The number of EAP Request frames (other than Rq/Id frames) that have been received by this Supplicant.

InvalidEapolFramesReceived

The number of EAPOL frames that have been received by this Supplicant in which the frame type is not recognized.

EapLengthErrorFramesReceived

The number of EAPOL frames that have been received by this Supplicant in which the Packet Body Length field (7.5.5) is invalid.

LastEapolFrameVersion

The protocol version number carried in the most recently received EAPOL frame.

LastEapolFrameSource

The source MAC address carried in the most recently received EAPOL frame.

Description

The *GetSupplicantStatistics()* function reads the statistical information *Statistics* regarding the operation of the Supplicant associated with the Port.

If *Statistics* is *NULL*, then *EFI_INVALID_PARAMETER* is returned.

Status Codes Returned

EFI_SUCCESS	The statistical information regarding the operation of the Supplicant for the Port is read successfully.
EFI_INVALID_PARAMETER	<i>Statistics</i> is <i>NULL</i> .

27.2.14 EFI EAP Management2 Protocol

27.2.14.1 EFI_EAP_MANAGEMENT2_PROTOCOL

Summary

This protocol provides the ability to configure and control EAPOL state machine, and retrieve the information, status and the statistics information of EAPOL state machine.

GUID


```
#define EFI_EAP_MANAGEMENT2_PROTOCOL_GUID \
{ 0x5e93c847, 0x456d, 0x40b3, \
  { 0xa6, 0xb4, 0x78, 0xb0, 0xc9, 0xcf, 0x7f, 0x20 } }
```

Protocol Interface Structure

```
typedef struct _EFI_EAP_MANAGEMENT2_PROTOCOL {
    ..... // Same as EFI_EAP_MANAGEMENT_PROTOCOL
    EFI_EAP_GET_KEY GetKey;
} EFI_EAP_MANAGEMENT2_PROTOCOL;
```

Parameters

GetKey

Provide Key information parsed from EAP packet. See the *GetKey()* function description.

Description

The *EFI_EAP_MANAGEMENT2_PROTOCOL* is used to control, configure and monitor EAPOL state machine on a Port, and return information of the Port. EAPOL state machine is built on a per-Port basis. Herein, a Port means a NIC. For the details of EAPOL, please refer to IEEE 802.1x specification.

27.2.15 EFI_EAP_MANAGEMENT2_PROTOCOL.GetKey()

Summary

Return key generated through EAP process.

Prototype

```
typedef
EFI_STATUS
(EFI_API *EFI_EAP_GET_KEY) (
    IN EFI_EAP_MANAGEMENT2_PROTOCOL *This,
    IN OUT UINT8 *Msk,
    IN OUT UINTN *MskSize,
    IN OUT UINT8 *Emsk,
    IN OUT UINT8 *EmskSize
);
```

Parameters

This

Pointer to the *EFI_EAP_MANAGEMENT2_PROTOCOL* instance.

Msk

Pointer to MSK (Master Session Key) buffer.

MskSize

MSK buffer size.

Emsk

Pointer to EMSK (Extended Master Session Key) buffer.

EmskSize

EMSK buffer size.

Description

The *GetKey()* function return the key generated through EAP process, so that the 802.11 MAC layer driver can use MSK to derive more keys, e.g. PMK (Pairwise Master Key).

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>Msk</i> is NULL. • <i>MskSize</i> is NULL. • <i>Emsk</i> is NULL. • <i>EmskSize</i> is NULL.
EFI_NOT_READY	MSK and EMSK are not generated in current session yet.

27.2.16 EFI EAP Configuration Protocol**27.2.16.1 EFI_EAP_CONFIGURATION_PROTOCOL****Summary**

This protocol provides a way to set and get EAP configuration.

GUID

```
#define EFI_EAP_CONFIGURATION_PROTOCOL_GUID \
{ 0xe5b58dbb, 0x7688, 0x44b4, \
  { 0x97, 0xbf, 0x5f, 0x1d, 0x4b, 0x7c, 0xc8, 0xdb } }
```

Protocol Interface Structure

```
typedef struct _EFI_EAP_CONFIGURATION_PROTOCOL {
    EFI_EAP_CONFIGURATION_SET_DATA      SetData;
    EFI_EAP_CONFIGURATION_GET_DATA      GetData;
} EFI_EAP_CONFIGURATION_PROTOCOL;
```

Parameters**SetData**

Set EAP configuration data. See the *SetData()* function description.

GetData

Get EAP configuration data. See the *GetData()* function description.

Description

The *EFI_EAP_CONFIGURATION_PROTOCOL* is designed to provide a way to set and get EAP configuration, such as Certificate, private key file.

27.2.17 EFI_EAP_CONFIGURATION_PROTOCOL.SetData()

Summary

Set EAP configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_EAP_CONFIGURATION_SET_DATA)(
    IN EFI_EAP_CONFIGURATION_PROTOCOL    *This,
    IN EFI_EAP_TYPE                      EapType,
    IN EFI_EAP_CONFIG_DATA_TYPE          DataType,
    IN VOID                             *Data,
    IN UINTN                             DataSize
);
```

Parameters

This

Pointer to the *EFI_EAP_CONFIGURATION_PROTOCOL* instance.

EapType

EAP type. See *EFI_EAP_TYPE*.

DataType

Configuration data type. See *EFI_EAP_CONFIG_DATA_TYPE*

Data

Pointer to configuration data.

DataSize

Total size of configuration data.

Description

The *SetData()* function sets EAP configuration to non-volatile storage or volatile storage.

Related Definitions

```
//
// Make sure it not conflict with any real EapTypeXXX
//
#define EFI_EAP_TYPE_ATTRIBUTE 0

typedef enum {
    // EFI_EAP_TYPE_ATTRIBUTE
    EfiEapConfigEapAuthMethod,
    EfiEapConfigEapSupportedAuthMethod,
    // EapTypeIdentity
    EfiEapConfigIdentityString,
    // EapTypeEAPTLS/EapTypePEAP
    EfiEapConfigEapTlsCACert,
    EfiEapConfigEapTlsClientCert,
    EfiEapConfigEapTlsClientPrivateKeyFile,
    EfiEapConfigEapTlsClientPrivateKeyFilePassword, \\
    // ASCII format, Volatile
```

(continues on next page)

(continued from previous page)

```

    EfiEapConfigEapTlsCipherSuite,
    EfiEapConfigEapTlsSupportedCipherSuite,
    // EapTypeMSChapV2
    EfiEapConfigEapMSChapV2Password, // UNICODE format, Volatile
    // EapTypePEAP
    EfiEapConfigEap2ndAuthMethod,
    // More...
} EFI_EAP_CONFIG_DATA_TYPE;

//
// EFI_EAP_TYPE
//
typedef UINT8 EFI_EAP_TYPE;
#define EFI_EAP_TYPE_ATTRIBUTE      0
#define EFI_EAP_TYPE_IDENTITY      1
#define EFI_EAP_TYPE_NOTIFICATION  2
#define EFI_EAP_TYPE_NAK           3
#define EFI_EAP_TYPE_MD5CHALLENGE  4
#define EFI_EAP_TYPE_OTP           5
#define EFI_EAP_TYPE_GTC           6
#define EFI_EAP_TYPE_EAPTLS        13
#define EFI_EAP_TYPE_EAPSIM        18
#define EFI_EAP_TYPE_TTLS          21
#define EFI_EAP_TYPE_PEAP          25
#define EFI_EAP_TYPE_MSCHAPV2      26
#define EFI_EAP_TYPE_EAP_EXTENSION 33
.....

```

Status Codes Returned

EFI_SUCCESS	The EAP configuration data is set successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>Data</i> is NULL. • <i>DataSize</i> is 0.
EFI_UNSUPPORTED	The <i>EapType</i> or <i>DataType</i> is unsupported.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

27.2.18 EFI_EAP_CONFIGURATION_PROTOCOL.GetData()**Summary**

Get EAP configuration data.

Prototype

```

typedef
EFI_STATUS
(EFIAPI *EFI_EAP_CONFIGURATION_GET_DATA)(

```

(continues on next page)

(continued from previous page)

```

IN EFI_EAP_CONFIGURATION_PROTOCOL    *This,
IN EFI_EAP_TYPE                      EapType,
IN EFI_EAP_CONFIG_DATA_TYPE          DataType,
IN OUT VOID                          *Data,
IN OUT UINTN                         *DataSize
);

```

Parameters**This**

Pointer to the *EFI_EAP_CONFIGURATION_PROTOCOL* instance.

EapType

EAP type. See *EFI_EAP_TYPE*.

DataType

Configuration data type. See *EFI_EAP_CONFIG_DATA_TYPE*

Data

Pointer to configuration data.

DataSize

Total size of configuration data. On input, it means the size of *Data * buffer*. On output, it means the size of copied **Data* buffer if *EFI_SUCCESS*, and means the size of desired *Data* buffer if *EFI_BUFFER_TOO_SMALL*.

Description

The *GetData()* function gets EAP configuration.

Status Codes Returned

EFI_SUCCESS	The EAP configuration data is got successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>Data</i> is NULL. • <i>DataSize</i> is NULL.
EFI_UNSUPPORTED	The <i>EapType</i> or <i>DataType</i> is unsupported.
EFI_NOT_FOUND	The EAP configuration data is not found.
EFI_BUFFER_TOO_SMALL	The buffer is too small to hold the buffer.

27.3 EFI Wireless MAC Connection Protocol

27.3.1 EFI_WIRELESS_MAC_CONNECTION_PROTOCOL

Summary

This protocol provides management service interfaces of 802.11 MAC layer. It is used by network applications (and drivers) to establish wireless connection with an access point (AP).

GUID

```
#define EFI_WIRELESS_MAC_CONNECTION_PROTOCOL_GUID \
{ 0xda55bc9, 0x45f8, 0x4bb4, \
  { 0x87, 0x19, 0x52, 0x24, 0xf1, 0x8a, 0x4d, 0x45 }}
```

Protocol Interface Structure

```
typedef struct _EFI_WIRELESS_MAC_CONNECTION_PROTOCOL {
    EFI_WIRELESS_MAC_CONNECTION_SCAN          Scan;
    EFI_WIRELESS_MAC_CONNECTION_ASSOCIATE      Associate;
    EFI_WIRELESS_MAC_CONNECTION_DISASSOCIATE    Disassociate;
    EFI_WIRELESS_MAC_CONNECTION_AUTHENTICATE    Authenticate;
    EFI_WIRELESS_MAC_CONNECTION_DEAUTHENTICATE  Deauthenticate;
} EFI_WIRELESS_MAC_CONNECTION_PROTOCOL;
```

Parameters

Scan

Determine the characteristics of the available BSSs. See the *Scan()* function description.

Associate

Places an association request with a specific peer MAC entity. See the *Associate()* function description.

Disassociate

Reports a disassociation with a specific peer MAC entity. See the *Disassociate()* function description.

Authenticate

Requests authentication with a specific peer MAC entity. See the *Authenticate()* function description.

Deauthenticate

Invalidates an authentication relationship with a peer MAC entity. See the *Deauthenticate()* function description.

Description

The *EFI_WIRELESS_MAC_CONNECTION_PROTOCOL* is designed to provide management service interfaces for the EFI wireless network stack to establish wireless connection with AP. An EFI Wireless MAC Connection Protocol instance will be installed on each communication device that the EFI wireless network stack runs on.

27.3.2 EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Scan()

Summary

Request a survey of potential BSSs that administrator can later elect to try to join.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_WIRELESS_MAC_CONNECTION_SCAN)(
    IN EFI_WIRELESS_MAC_CONNECTION_PROTOCOL    *This,
    IN EFI_80211_SCAN_DATA_TOKEN                *Data
);
```

Parameters

This

Pointer to the *EFI_WIRELESS_MAC_CONNECTION_PROTOCOL*

Data

Pointer to the scan token. Type `EFI_80211_SCAN_DATA_TOKEN` is defined in “Related Definitions” below.

Description

The `Scan()` function returns the description of the set of BSSs detected by the scan process. Passive scan operation is performed by default.

Related Definitions

```
//*****
// EFI_80211_SCAN_DATA_TOKEN
//*****
typedef struct {
    EFI_EVENT                Event;
    EFI_STATUS               Status;
    EFI_80211_SCAN_DATA     *Data;
    EFI_80211_SCAN_RESULT_CODE ResultCode;
    EFI_80211_SCAN_RESULT   *Result;
} EFI_80211_SCAN_DATA_TOKEN;
```

Event

This *Event* will be signaled after the *Status* field is updated by the EFI Wireless MAC Connection Protocol driver. The type of *Event* must be `EFI_NOTIFY_SIGNAL`.

Status

Will be set to one of the following values:

EFI_SUCCESS: Scan operation completed successfully.

EFI_NOT_FOUND: Failed to find available BSS.

EFI_DEVICE_ERROR: An unexpected network or system error occurred.

EFI_ACCESS_DENIED: The scan operation is not completed due to some underlying hardware or software state.

EFI_NOT_READY: The scan operation is started but not yet completed.

Data

Pointer to the scan data. Type `EFI_80211_SCAN_DATA` is defined below.

ResultCode

Indicates the scan state. Type `EFI_80211_SCAN_RESULT_CODE` is defined below.

Result

Indicates the scan result. It is caller’s responsibility to free this buffer. Type `EFI_80211_SCAN_RESULT` is defined below.

The `EFI_80211_SCAN_DATA_TOKEN` structure is defined to support the process of determining the characteristics of the available BSSs. As input, the *Data* field must be filled in by the caller of EFI Wireless MAC Connection Protocol. After the scan operation completes, the EFI Wireless MAC Connection Protocol driver updates the *Status*, *ResultCode* and *Result* field and the *Event* is signaled.

```
//*****
// EFI_80211_SCAN_DATA
//*****
```

(continues on next page)

(continued from previous page)

```
typedef struct {
    EFI_80211_BSS_TYPE          BSSType;
    EFI_80211_MAC_ADDRESS       BSSId;
    UINT8                       SSIdLen;
    UINT8                       *SSId;
    BOOLEAN                     PassiveMode;
    UINT32                      ProbeDelay;
    UINT32                      *ChannelList;
    UINT32                      MinChannelTime;
    UINT32                      MaxChannelTime;
    EFI_80211_ELEMENT_REQ       *RequestInformation;
    EFI_80211_ELEMENT_SSID      *SSIDList;
    EFI_80211_ACC_NET_TYPE      AccessNetworkType;
    UINT8                       *VendorSpecificInfo;
} EFI_80211_SCAN_DATA;
```

BSSType

Determines whether infrastructure BSS, IBSS, MBSS, or all, are included in the scan. Type EFI_80211_BSS_TYPE is defined below.

BSSId

Indicates a specific or wildcard BSSID. Use all binary 1s to represent all SSIDs. Type EFI_80211_MAC_ADDRESS is defined below.

SSIdLen

Length in bytes of the SSId. If zero, ignore the SSId field.

SSId

Specifies the desired SSID or the wildcard SSID. Use NULL to represent all SSIDs.

PassiveMode

Indicates passive scanning if TRUE.

ProbeDelay

The delay in microseconds to be used prior to transmitting a Probe frame during active scanning. If zero, the value can be overridden by an implementation-dependent default value.

ChannelList

Specifies a list of channels that are examined when scanning for a BSS. If set to NULL, all valid channels will be scanned.

MinChannelTime

Indicates the minimum time in TU to spend on each channel when scanning. If zero, the value can be overridden by an implementation-dependent default value.

MaxChannelTime

Indicates the maximum time in TU to spend on each channel when scanning. If zero, the value can be overridden by an implementation-dependent default value.

RequestInformation

Points to an optionally present element. This is an optional parameter and may be NULL. Type EFI_80211_ELEMENT_REQ is defined below.

SSIDList

Indicates one or more SSID elements that are optionally present. This is an optional parameter and may be NULL. Type EFI_80211_ELEMENT_SSID is defined below.

AccessNetworkType

Specifies a desired specific access network type or the wildcard access network type. Use 15 as wildcard access network type. Type EFI_80211_ACC_NET_TYPE is defined below.

VendorSpecificInfo

Specifies zero or more elements. This is an optional parameter and may be NULL.

```

//*****
// EFI_80211_BSS_TYPE
//*****
typedef enum {
    IeeeInfrastructureBSS,
    IeeeIndependentBSS,
    IeeeMeshBSS,
    IeeeAnyBss
} EFI_80211_BSS_TYPE;

```

The EFI_80211_BSS_TYPE is defined to enumerate BSS type.

```

//*****
// EFI_80211_MAC_ADDRESS
//*****
typedef struct {
    UINT8 Addr[6];
} EFI_80211_MAC_ADDRESS;

```

The EFI_80211_MAC_ADDRESS is defined to record a 48-bit MAC address.

```

//*****
// EFI_80211_ELEMENT_REQ
//*****
typedef struct {
    EFI_80211_ELEMENT_HEADER    Hdr;
    UINT8                       RequestIDs[1];
} EFI_80211_ELEMENT_REQ;

```

Hdr

Common header of an element. Type EFI_80211_ELEMENT_HEADER is defined below.

RequestIDs

Start of elements that are requested to be included in the Probe Response frame. The elements are listed in order of increasing element ID.

```

//*****
// EFI_80211_ELEMENT_HEADER
//*****
typedef struct {
    UINT8        ElementID;
    UINT8        Length;
} EFI_80211_ELEMENT_HEADER;

```

ElementID

A unique element ID defined in IEEE 802.11 specification.

Length

Specifies the number of octets in the element body.

```

//*****
// EFI_80211_ELEMENT_SSID
//*****
typedef struct {
    EFI_80211_ELEMENT_HEADER    Hdr;
    UINT8                      SSId [32];
}    EFI_80211_ELEMENT_SSID;
    
```

Hdr

Common header of an element.

SSId

Service set identifier. If Hdr.Length is zero, this field is ignored.

```

//*****
// EFI_80211_ACC_NET_TYPE
//*****
typedef enum {
    IeeePrivate = 0,
    IeeePrivatewithGuest = 1,
    IeeeChargeablePublic = 2,
    IeeeFreePublic = 3,
    IeeePersonal = 4,
    IeeeEmergencyServOnly = 5,
    IeeeTestOrExp = 14,
    IeeeWildcard = 15
}    EFI_80211_ACC_NET_TYPE;
    
```

The *EFI_80211_ACC_NET_TYPE* records access network types defined in IEEE 802.11 specification.

```

//*****
// EFI_80211_SCAN_RESULT_CODE
//*****
typedef enum {
    ScanSuccess,
    ScanNotSupported
}    EFI_80211_SCAN_RESULT_CODE;
    
```

ScanSuccess

The scan operation finished successfully.

ScanNotSupported

The scan operation is not supported in current implementation.

```

//*****
// EFI_80211_SCAN_RESULT
//*****
typedef struct {
    UINTN                      NumOfBSSDesp;
    EFI_80211_BSS_DESCRIPTION **BSSDespSet;
    UINTN                      NumofBSSDespFromPilot;
    EFI_80211_BSS_DESP_PILOT  **BSSDespFromPilotSet;
    UINT8                      *VendorSpecificInfo;
}    EFI_80211_SCAN_RESULT;
    
```

NumOfBSSDesp

The number of EFI_80211_BSS_DESCRIPTION in BSSDespSet. If zero, BSSDespSet should be ignored.

BSSDespSet

Points to zero or more instances of EFI_80211_BSS_DESCRIPTION. Type EFI_80211_BSS_DESCRIPTION is defined below.

NumOfBSSDespFromPilot

The number of EFI_80211_BSS_DESP_PILOT in BSSDespFromPilotSet. If zero, BSSDespFromPilotSet should be ignored.

BSSDespFromPilotSet

Points to zero or more instances of EFI_80211_BSS_DESP_PILOT. Type EFI_80211_BSS_DESP_PILOT is defined below.

VendorSpecificInfo

Specifies zero or more elements. This is an optional parameter and may be NULL.

```
//*****
// EFI_80211_BSS_DESCRIPTION
//*****
typedef struct {
    EFI_80211_MAC_ADDRESS *BSSId;
    UINT8                  *SSId;
    UINT8                  SSIdLen;
    EFI_80211_BSS_TYPE      BSSType;
    UINT16                  BeaconPeriod;
    UINT64                  Timestamp;
    UINT16                  CapabilityInfo;
    UINT8                  *BSSBasicRateSet;
    UINT8                  *OperationalRateSet;
    EFI_80211_ELEMENT_COUNTRY *Country;
    EFI_80211_ELEMENT_RSN    RSN;
    UINT8                  RSSI;
    UINT8                  RCPIMeasurement;
    UINT8                  RSNIMeasurement;
    UINT8                  *RequestedElements;
    UINT8                  *BSSMembershipSelectorSet;
    EFI_80211_ELEMENT_EXT_CAP *ExtCapElement;
} EFI_80211_BSS_DESCRIPTION;
```

BSSId

Indicates a specific BSSID of the found BSS.

SSId

Specifies the SSID of the found BSS. If NULL, ignore SSIdLen field.

SSIdLen

Length in bytes of the SSId. If zero, ignore SSId field.

BSSType

Specifies the type of the found BSS.

BeaconPeriod

The beacon period in TU of the found BSS.

Timestamp

The timestamp of the received frame from the found BSS.

CapabilityInfo

The advertised capabilities of the BSS.

BSSBasicRateSet

The set of data rates that shall be supported by all STAs that desire to join this BSS.

OperationalRateSet

The set of data rates that the peer STA desires to use for communication within the BSS.

Country

The information required to identify the regulatory domain in which the peer STA is located. Type EFI_80211_ELEMENT_COUNTRY is defined below.

RSN

The cipher suites and AKM suites supported in the BSS. Type EFI_80211_ELEMENT_RSN is defined below.

RSSI

Specifies the RSSI of the received frame.

RCPIMeasurement

Specifies the RCPI of the received frame.

RSNIMeasurement

Specifies the RSNI of the received frame.

RequestedElements

Specifies the elements requested by the request element of the Probe Request frame. This is an optional parameter and may be NULL.

BSSMembershipSelectorSet

Specifies the BSS membership selectors that represent the set of features that shall be supported by all STAs to join this BSS.

ExtCapElement

Specifies the parameters within the Extended Capabilities element that are supported by the MAC entity. This is an optional parameter and may be NULL. Type EFI_80211_ELEMENT_EXT_CAP is defined below.

```

//*****
// EFI_80211_ELEMENT_COUNTRY
//*****
typedef struct {
    EFI_80211_ELEMENT_HEADER    Hdr;
    UINT8                      CountryStr [3];
    EFI_80211_COUNTRY_TRIPLET   CountryTriplet[1];
}    EFI_80211_ELEMENT_COUNTRY;

```

Hdr

Common header of an element.

CountryStr

Specifies country strings in 3 octets.

CountryTriplet

Indicates a triplet that repeated in country element. The number of triplets is determined by the Hdr.Length field.

```

//*****
// EFI_80211_COUNTRY_TRIPLET
//*****
typedef union {
    EFI_80211_COUNTRY_TRIPLET_SUBBAND    Subband;

```

(continues on next page)

(continued from previous page)

```
EFI_80211_COUNTRY_TRIPLET_OPERATE    Operating;
}   EFI_80211_COUNTRY_TRIPLET;
```

Subband

The subband triplet.

Operating

The operating triplet.

```
/**
 * EFI_80211_COUNTRY_TRIPLET_SUBBAND
 */
typedef struct {
    UINT8          FirstChannelNum;
    UINT8          NumOfChannels;
    UINT8          MaxTxPowerLevel;
}   EFI_80211_COUNTRY_TRIPLET_SUBBAND;
```

FirstChannelNum

Indicates the lowest channel number in the subband. It has a positive integer value less than 201.

NumOfChannels

Indicates the number of channels in the subband.

MaxTxPowerLevel

Indicates the maximum power in dBm allowed to be transmitted.

```
/**
 * EFI_80211_COUNTRY_TRIPLET_OPERATE
 */
typedef struct {
    UINT8          OperatingExtId;
    UINT8          OperatingClass;
    UINT8          CoverageClass;
}   EFI_80211_COUNTRY_TRIPLET_OPERATE;
```

OperatingExtId

Indicates the operating extension identifier. It has a positive integer value of 201 or greater.

OperatingClass

Index into a set of values for radio equipment set of rules.

CoverageClass

Specifies an AirPropagationTime characteristics used in BSS operation. Refer the definition of an AirPropagationTime in IEEE 802.11 specification.

```
/**
 * EFI_80211_ELEMENT_RSN
 */
typedef struct {
    EFI_80211_ELEMENT_HEADER    Hdr;
    EFI_80211_ELEMENT_DATA_RSN *Data;
}   EFI_80211_ELEMENT_RSN;
```

Hdr

Common header of an element.

Data

Points to RSN element. Type EFI_80211_ELEMENT_DATA_RSN is defined below. The size of a RSN element is limited to 255 octets.

```
//*****
// EFI_80211_ELEMENT_DATA_RSN
//*****
typedef struct {
    UINT16      Version;
    UINT32      GroupDataCipherSuite;
//UINT16      PairwiseCipherSuiteCount;
//UINT32      PairwiseCipherSuiteList [PairwiseCipherSuiteCount];
//UINT16      AKMSuiteCount;
//UINT32      AKMSuiteList [AKMSuiteCount];
//UINT16      RSNCapabilities;
//UINT16      PMKIDCount;
//UINT8       PMKIDList [PMKIDCount][16];
//UINT32      GroupManagementCipherSuite;
} EFI_80211_ELEMENT_DATA_RSN;
```

Version

Indicates the version number of the RSNA protocol. Value 1 is defined in current IEEE 802.11 specification.

GroupDataCipherSuite

Specifies the cipher suite selector used by the BSS to protect group address frames.

PairwiseCipherSuiteCount

Indicates the number of pairwise cipher suite selectors that are contained in PairwiseCipherSuiteList.

PairwiseCipherSuiteList

Contains a series of cipher suite selectors that indicate the pairwise cipher suites contained in this element.

AKMSuiteCount

Indicates the number of AKM suite selectors that are contained in AKMSuiteList.

AKMSuiteList

Contains a series of AKM suite selectors that indicate the AKM suites contained in this element.

RSNCapabilities

Indicates requested or advertised capabilities.

PMKIDCount

Indicates the number of PKMIDs in the PMKIDList.

PMKIDList

Contains zero or more PKMIDs that the STA believes to be valid for the destination AP.

GroupManagementCipherSuite

Specifies the cipher suite selector used by the BSS to protect group addressed robust management frames.

```
//*****
// EFI_80211_ELEMENT_EXT_CAP
//*****
typedef struct {
    EFI_80211_ELEMENT_HEADER    Hdr;
    UINT8                       Capabilities[1];
} EFI_80211_ELEMENT_EXT_CAP;
```

Hdr

Common header of an element.

Capabilities

Indicates the capabilities being advertised by the STA transmitting the element. This is a bit field with variable length. Refer to IEEE 802.11 specification for bit value.

```
//*****
// EFI_80211_BSS_DESP_PILOT
//*****
typedef struct {
    EFI_80211_MAC_ADDRESS    BSSId;
    EFI_80211_BSS_TYPE       BSSType;
    UINT8                    ConCapInfo;
    UINT8                    ConCountryStr[2];
    UINT8                    OperatingClass;
    UINT8                    Channel;
    UINT8                    Interval;
    EFI_80211_MULTIPLE_BSSID *MultipleBSSID;
    UINT8                    RCPIMeasurement;
    UINT8                    RSNIMeasurement;
} EFI_80211_BSS_DESP_PILOT;
```

BSSId

Indicates a specific BSSID of the found BSS.

BSSType

Specifies the type of the found BSS.

ConCapInfo

One octet field to report condensed capability information.

ConCountryStr

Two octet's field to report condensed country string.

OperatingClass

Indicates the operating class value for the operating channel.

Channel

Indicates the operating channel.

Interval

Indicates the measurement pilot interval in TU.

MultipleBSSID

Indicates that the BSS is within a multiple BSSID set.

RCPIMeasurement

Specifies the RCPI of the received frame.

RSNIMeasurement

Specifies the RSNI of the received frame.

```
//*****
// EFI_80211_MULTIPLE_BSSID
//*****
typedef struct {
    EFI_80211_ELEMENT_HEADER    Hdr;
    UINT8                      Indicator;
```

(continues on next page)

(continued from previous page)

```
EFI_80211_SUBELEMENT_INFO    SubElement[1];
}    EFI_80211_MULTIPLE_BSSID;
```

Hdr

Common header of an element.

Indicator

Indicates the maximum number of BSSIDs in the multiple BSSID set. When Indicator is set to n, 2n is the maximum number.

SubElement

Contains zero or more sub-elements. Type EFI_80211_SUBELEMENT_INFO is defined below.

```
/**
 * *****
 */
// EFI_80211_SUBELEMENT_INFO
// *****
typedef struct {
    UINT8        SubElementID;
    UINT8        Length;
    UINT8        Data[1];
}    EFI_80211_SUBELEMENT_INFO;
```

SubElementID

Indicates the unique identifier within the containing element or sub-element.

Length

Specifies the number of octets in the Data field.

Data

A variable length data buffer.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>This</i> is NULL. • <i>Data</i> is NULL. • <i>Data</i> -> <i>Data</i> is NULL.
EFI_UNSUPPORTED	One or more of the input parameters are not supported by this implementation.
EFI_ALREADY_STARTED	The scan operation is already started.

27.3.3 EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Associate()

Summary

Request an association with a specified peer MAC entity that is within an AP.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_WIRELESS_MAC_CONNECTION_ASSOCIATE)(
    IN EFI_WIRELESS_MAC_CONNECTION_PROTOCOL      *This,
    IN EFI_80211_ASSOCIATE_DATA_TOKEN            *Data
);
```

Parameters

This

Pointer to the *EFI_WIRELESS_MAC_CONNECTION_PROTOCOL* instance.

Data

Pointer to the association token. Type *EFI_80211_ASSOCIATE_DATA_TOKEN* is defined in Related Definitions below.

Description

The *Associate()* function provides the capability for MAC layer to become associated with an AP.

Related Definitions

```
/**
//*****
// EFI_80211_ASSOCIATE_DATA_TOKEN
//*****
typedef struct {
    EFI_EVENT              Event;
    EFI_STATUS             Status;
    EFI_80211_ASSOCIATE_DATA Data;
    EFI_80211_ASSOCIATE_RESULT_CODE ResultCode;
    EFI_80211_ASSOCIATE_RESULT Result;
} EFI_80211_ASSOCIATE_DATA_TOKEN;
```

Event

This Event will be signaled after the Status field is updated by the EFI Wireless MAC Connection Protocol driver. The type of Event must be *EFI_NOTIFY_SIGNAL*.

Status

Will be set to one of the following values: *EFI_SUCCESS*: Association operation completed successfully. *EFI_DEVICE_ERROR*: An unexpected network or system error occurred.

Data

Pointer to the association data. Type *EFI_80211_ASSOCIATE_DATA* is defined below.

ResultCode

Indicates the association state. Type *EFI_80211_ASSOCIATE_RESULT_CODE* is defined below.

Result

Indicates the association result. It is caller's responsibility to free this buffer. Type *EFI_80211_ASSOCIATE_RESULT** is defined below.

The `EFI_80211_ASSOCIATE_DATA_TOKEN` structure is defined to support the process of association with a specified AP. As input, the *Data* field must be filled in by the caller of EFI Wireless MAC Connection Protocol. After the association operation completes, the EFI Wireless MAC Connection Protocol driver updates the *Status*, *ResultCode* and *Result* field and the *Event* is signaled.

```

//*****
// EFI_80211_ASSOCIATE_DATA
//*****
typedef struct {
    EFI_80211_MAC_ADDRESS      BSSId;
    UINT16                     CapabilityInfo;
    UINT32                     FailureTimeout;
    UINT32                     ListenInterval;
    EFI_80211_ELEMENT_SUPP_CHANNEL *Channels;
    EFI_80211_ELEMENT_RSN      RSN;
    EFI_80211_ELEMENT_EXT_CAP  ExtCapElement;
    UINT8                      *VendorSpecificInfo;
} EFI_80211_ASSOCIATE_DATA;

```

BSSId

Specifies the address of the peer MAC entity to associate with.

CapabilityInfo

Specifies the requested operational capabilities to the AP in 2 octets.

FailureTimeout

Specifies a time limit in TU, after which the associate procedure is terminated.

ListenInterval

Specifies if in power save mode, how often the STA awakes and listens for the next beacon frame in TU.

Channels

Indicates a list of channels in which the STA is capable of operating. Type `EFI_80211_ELEMENT_SUPP_CHANNEL` is defined below.

RSN

The cipher suites and AKM suites selected by the STA.

ExtCapElement

Specifies the parameters within the Extended Capabilities element that are supported by the MAC entity. This is an optional parameter and may be NULL.

VendorSpecificInfo

Specifies zero or more elements. This is an optional parameter and may be NULL.

```

//*****
// EFI_80211_ELEMENT_SUPP_CHANNEL
//*****
typedef struct {
    EFI_80211_ELEMENT_HEADER      Hdr;
    EFI_80211_ELEMENT_SUPP_CHANNEL_TUPLE Subband[1];
} EFI_80211_ELEMENT_SUPP_CHANNEL;

```

Hdr

Common header of an element.

Subband

Indicates one or more tuples of (first channel, number of channels). Type

EFI_80211_ELEMENT_SUPP_CHANNEL_TUPLE is defined below.

```

//*****
// EFI_80211_ELEMENT_SUPP_CHANNEL_TUPLE
//*****
typedef struct {
    UINT8      FirstChannelNumber;
    UINT8      NumberOfChannels;
}   EFI_80211_ELEMENT_SUPP_CHANNEL_TUPLE;
    
```

FirstChannelNumber

The first channel number in a subband of supported channels.

NumberOfChannels

The number of channels in a subband of supported channels.

```

//*****
// EFI_80211_ASSOCIATE_RESULT_CODE
//*****
typedef enum {
    AssociateSuccess,
    AssociateRefusedReasonUnspecified,
    AssociateRefusedCapsMismatch,
    AssociateRefusedExtReason,
    AssociateRefusedAPOutOfMemory,
    AssociateRefusedBasicRatesMismatch,
    AssociateRejectedEmergencyServicesNotSupported,
    AssociateRefusedTemporarily
}   EFI_80211_ASSOCIATE_RESULT_CODE;
    
```

The *EFI_80211_ASSOCIATE_RESULT_CODE* records the result responses to the association request, which are defined in IEEE 802.11 specification.

```

//*****
// EFI_80211_ASSOCIATE_RESULT
//*****
typedef struct {
    EFI_80211_MAC_ADDRESS      BSSId;
    UINT16                     CapabilityInfo;
    UINT16                     AssociationID;
    UINT8                      RCPIValue;
    UINT8                      RSNIValue;
    EFI_80211_ELEMENT_EXT_CAP  *ExtCapElement;
    EFI_80211_ELEMENT_TIMEOUT_VAL TimeoutInterval;
    UINT8                      *VendorSpecificInfo;
}   EFI_80211_ASSOCIATE_RESULT;
    
```

BSSId

Specifies the address of the peer MAC entity from which the association request was received.

CapabilityInfo

Specifies the operational capabilities advertised by the AP.

AssociationID

Specifies the association ID value assigned by the AP.

RCPIValue

Indicates the measured RCPI of the corresponding association request frame. It is an optional parameter and is set to zero if unavailable.

RSNValue

Indicates the measured RSN at the time the corresponding association request frame was received. It is an optional parameter and is set to zero if unavailable.

ExtCapElement

Specifies the parameters within the Extended Capabilities element that are supported by the MAC entity. This is an optional parameter and may be NULL.

TimeoutInterval

Specifies the timeout interval when the result code is AssociateRefusedTemporarily .

VendorSpecificInfo

Specifies zero or more elements. This is an optional parameter and may be NULL.

```

//*****
// EFI_80211_ELEMENT_TIMEOUT_VAL
//*****
typedef struct {
    EFI_80211_ELEMENT_HEADER    Hdr;
    UINT8                       Type;
    UINT32                      Value;
} EFI_80211_ELEMENT_TIMEOUT_VAL;

```

Hdr

Common header of an element.

Type

Specifies the timeout interval type.

Value

Specifies the timeout interval value.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>This</i> is NULL. • <i>Data</i> is NULL. • <i>Data</i> -> <i>Data</i> is NULL.
EFI_UNSUPPORTED	One or more of the input parameters are not supported by this implementation.
EFI_ALREADY_STARTED	The association process is already started.
EFI_NOT_READY	Authentication is not performed before this association process.
EFI_NOT_FOUND	The specified peer MAC entity is not found.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

27.3.4 EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Disassociate()

Summary

Request a disassociation with a specified peer MAC entity.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_WIRELESS_MAC_CONNECTION_DISASSOCIATE)(
    IN EFI_WIRELESS_MAC_CONNECTION_PROTOCOL      *This,
    IN EFI_80211_DISASSOCIATE_DATA_TOKEN         *Data
);
```

Parameters

This

Pointer to the *EFI_WIRELESS_MAC_CONNECTION_PROTOCOL* instance.

Data

Pointer to the disassociation token. Type *EFI_80211_DISASSOCIATE_DATA_TOKEN* is defined in Related Definitions below.

Description

The *Disassociate()* function is invoked to terminate an existing association. Disassociation is a notification and cannot be refused by the receiving peer except when management frame protection is negotiated and the message integrity check fails.

Related Definitions

```
/**
//*****
//  EFI_80211_DISASSOCIATE_DATA_TOKEN
//*****
typedef struct {
    EFI_EVENT              Event;
    EFI_STATUS             Status;
    EFI_80211_DISASSOCIATE_DATA *Data;
    EFI_80211_DISASSOCIATE_RESULT_CODE ResultCode;
} EFI_80211_DISASSOCIATE_DATA_TOKEN;
```

Event

This Event will be signaled after the Status field is updated by the EFI Wireless MAC Connection Protocol driver. The type of Event must be *EFI_NOTIFY_SIGNAL*.

Status

Will be set to one of the following values:

EFI_SUCCESS: Disassociation operation completed successfully.

EFI_DEVICE_ERROR: An unexpected network or system error occurred.

EFI_ACCESS_DENIED: The disassociation operation is not completed due to some underlying hardware or software state.

EFI_NOT_READY: The disassociation operation is started but not yet completed.

Data

Pointer to the disassociation data. Type EFI_80211_DISASSOCIATE_DATA is defined below.

ResultCode

Indicates the disassociation state. Type EFI_80211_DISASSOCIATE_RESULT_CODE is defined below.

```
//*****
// EFI_80211_DISASSOCIATE_DATA
//*****
typedef struct {
    EFI_80211_MAC_ADDRESS      BSSId;
    EFI_80211_REASON_CODE      ReasonCode;
    UINT8                      *VendorSpecificInfo;
} EFI_80211_DISASSOCIATE_DATA;
```

BSSId

Specifies the address of the peer MAC entity with which to perform the disassociation process.

ReasonCode

Specifies the reason for initiating the disassociation process.

VendorSpecificInfo

Zero or more elements, may be NULL.

```
//*****
// EFI_80211_REASON_CODE
//*****
typedef enum {
    Ieee80211UnspecifiedReason = 1,
    Ieee80211PreviousAuthenticateInvalid = 2,
    Ieee80211DeauthenticatedSinceLeaving = 3,
    Ieee80211DisassociatedDueToInactive = 4,
    Ieee80211DisassociatedSinceApUnable = 5,
    Ieee80211Class2FrameNonauthenticated = 6,
    Ieee80211Class3FrameNonassociated = 7,
    Ieee80211DisassociatedSinceLeaving = 8,
    // ...
} EFI_80211_REASON_CODE;
```

Note: The reason codes are defined in chapter 8.4.1.7 Reason Code field, IEEE 802.11-2012.

```
//*****
// EFI_80211_DISASSOCIATE_RESULT_CODE
//*****
typedef enum {
    DisassociateSuccess,
    DisassociateInvalidParameters
} EFI_80211_DISASSOCIATE_RESULT_CODE;
```

DisassociateSuccess

Disassociation process completed successfully.

DisassociateInvalidParameters

Disassociation failed due to any input parameter is invalid.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>This</i> is NULL. • <i>Data</i> is NULL.
EFI_ALREADY_STARTED	The disassociation process is already started.
EFI_NOT_READY	The disassociation service is invoked to a nonexistent association relationship.
EFI_NOT_FOUND	The specified peer MAC entity is not found.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

27.3.5 EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Authenticate()

Summary

Request the process of establishing an authentication relationship with a peer MAC entity.

Prototype

```
typedef
EFI_STATUS
(EFI_API *EFI_WIRELESS_MAC_CONNECTION_AUTHENTICATE) (
    IN EFI_WIRELESS_MAC_CONNECTION_PROTOCOL      *This,
    IN EFI_80211_AUTHENTICATE_DATA_TOKEN         *Data
);
```

Parameters

This

Pointer to the *EFI_WIRELESS_MAC_CONNECTION_PROTOCOL* instance.

Data

Pointer to the authentication token. Type *EFI_80211_AUTHENTICATE_DATA_TOKEN* is defined in Related Definitions below.

Description

The *Authenticate()* function requests authentication with a specified peer MAC entity. This service might be time-consuming thus is designed to be invoked independently of the association service.

Related Definitions

```
/**
//*****
//  EFI_80211_AUTHENTICATE_DATA_TOKEN
//*****
typedef struct {
    EFI_EVENT              Event;
    EFI_STATUS             Status;
    EFI_80211_AUTHENTICATE_DATA *Data;
    EFI_80211_AUTHENTICATE_RESULT_CODE ResultCode;
    EFI_80211_AUTHENTICATE_RESULT Result;
} EFI_80211_AUTHENTICATE_DATA_TOKEN;
```

Event

This Event will be signaled after the Status field is updated by the EFI Wireless MAC Connection Protocol driver. The type of Event must be EFI_NOTIFY_SIGNAL.

Status

Will be set to one of the following values:

EFI_PROTOCOL_ERROR: Peer MAC entity rejects the authentication.

EFI_NO_RESPONSE: Peer MAC entity does not response the authentication request.

EFI_DEVICE_ERROR: An unexpected network or system error occurred.

EFI_ACCESS_DENIED: The authentication operation is not completed due to some underlying hardware or software state.

EFI_NOT_READY: The authentication operation is started but not yet completed.

Data

Pointer to the authentication data. Type EFI_80211_AUTHENTICATE_DATA is defined below.

ResultCode

Indicates the association state. Type EFI_80211_AUTHENTICATE_RESULT_CODE is defined below.

Result

Indicates the association result. It is caller's responsibility to free this buffer. Type EFI_80211_AUTHENTICATE_RESULT is defined below.

```
//*****
// EFI_80211_AUTHENTICATION_DATA
//*****
typedef struct {
    EFI_80211_MAC_ADDRESS      BSSId;
    EFI_80211_AUTHENTICATION_TYPE AuthType;
    UINT32                     FailureTimeout;
    UINT8                      *FTContent;
    UINT8                      *SAEContent;
    UINT8                      *VendorSpecificInfo;
} EFI_80211_AUTHENTICATE_DATA;
```

BSSId

Specifies the address of the peer MAC entity with which to perform the authentication process.

AuthType

Specifies the type of authentication algorithm to use during the authentication process.

FailureTimeout

Specifies a time limit in TU after which the authentication procedure is terminated.

FTContent

Specifies the set of elements to be included in the first message of the FT authentication sequence, may be NULL.

SAEContent

Specifies the set of elements to be included in the SAE Commit Message or SAE Confirm Message, may be NULL.

VendorSpecificInfo

Zero or more elements, may be NULL.


```

//*****
// EFI_80211_AUTHENTICATION_TYPE
//*****
typedef enum {
    OpenSystem,
    SharedKey,
    FastBSSTransition,
    SAE
} EFI_80211_AUTHENTICATION_TYPE;

```

OpenSystem

Open system authentication, admits any STA to the DS.

SharedKey

Shared Key authentication relies on WEP to demonstrate knowledge of a WEP encryption key.

FastBSSTransition

FT authentication relies on keys derived during the initial mobility domain association to authenticate the stations.

SAE

SAE authentication uses finite field cryptography to prove knowledge of a shared password.

```

//*****
// EFI_80211_AUTHENTICATION_RESULT_CODE
//*****
typedef enum {
    AuthenticateSuccess,
    AuthenticateRefused,
    AuthenticateAnticLoggingTokenRequired,
    AuthenticateFiniteCyclicGroupNotSupported,
    AuthenticationRejected,
    AuthenticateInvalidParameter
} EFI_80211_AUTHENTICATE_RESULT_CODE;

```

The result code indicates the result response to the authentication request from the peer MAC entity.

```

//*****
// EFI_80211_AUTHENTICATION_RESULT
//*****
typedef struct {
    EFI_80211_MAC_ADDRESS    BSSId;
    UINT8                    *FTContent;
    UINT8                    *SAEContent;
    UINT8                    *VendorSpecificInfo;
} EFI_80211_AUTHENTICATE_RESULT;

```

BSSId

Specifies the address of the peer MAC entity from which the authentication request was received.

FTContent

Specifies the set of elements to be included in the second message of the FT authentication sequence, may be NULL.

SAEContent

Specifies the set of elements to be included in the SAE Commit Message or SAE Confirm Message, may be NULL.

VendorSpecificInfo

Zero or more elements, may be NULL.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>This</i> is NULL. • <i>Data</i> is NULL. • <i>Data Data</i> is NULL.
EFI_UNSUPPORTED	One or more of the input parameters are not supported by this implementation.
EFI_ALREADY_STARTED	The authentication process is already started.
EFI_NOT_FOUND	The specified peer MAC entity is not found.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

27.3.6 EFI_WIRELESS_MAC_CONNECTION_PROTOCOL.Deauthenticate()**Summary**

Invalidate the authentication relationship with a peer MAC entity.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_WIRELESS_MAC_CONNECTION_DEAUTHENTICATE) (
    IN EFI_WIRELESS_MAC_CONNECTION_PROTOCOL      *This,
    IN EFI_80211_DEAUTHENTICATE_DATA_TOKEN      *Data
);
```

Parameters**This**

Pointer to the *EFI_WIRELESS_MAC_CONNECTION_PROTOCOL* instance.

Data

Pointer to the deauthentication token. Type *EFI_80211_DEAUTHENTICATE_DATA_TOKEN* is defined in Related Definitions below.

Description

The *Deauthenticate()* function requests that the authentication relationship with a specified peer MAC entity be invalidated. Deauthentication is a notification and when it is sent out the association at the transmitting station is terminated.

Related Definitions

```
//*****
// EFI_80211_DEAUTHENTICATE_DATA_TOKEN
//*****
typedef struct {
    EFI_EVENT          Event;
    EFI_STATUS         Status;
```

(continues on next page)

(continued from previous page)

```
EFI_80211_DEAUTHENTICATE_DATA    *Data;
}   EFI_80211_DEAUTHENTICATE_DATA_TOKEN;
```

Event

This Event will be signaled after the Status field is updated by the EFI Wireless MAC Connection Protocol driver. The type of Event must be EFI_NOTIFY_SIGNAL.

Status

Will be set to one of the following values:

EFI_SUCCESS: Deauthentication operation completed successfully.

EFI_DEVICE_ERROR: An unexpected network or system error occurred.

EFI_ACCESS_DENIED: The deauthentication operation is not completed due to some underlying hardware or software state.

EFI_NOT_READY: The deauthentication operation is started but not yet completed.

Data

Pointer to the deauthentication data. Type EFI_80211_DEAUTHENTICATE_DATA is defined below.

```
/**
 * *****
 * // EFI_80211_DEAUTHENTICATE_DATA
 * *****
 */
typedef struct {
    EFI_80211_MAC_ADDRESS    BSSId;
    EFI_80211_REASON_CODE    ReasonCode;
    UINT8                    *VendorSpecificInfo;
}   EFI_80211_DEAUTHENTICATE_DATA;
```

BSSId

Specifies the address of the peer MAC entity with which to perform the deauthentication process.

ReasonCode

Specifies the reason for initiating the deauthentication process.

VendorSpecificInfo

Zero or more elements, may be NULL.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • This is NULL. • Data is NULL. • Data Data is NULL.
EFI_ALREADY_STARTED	The deauthentication process is already started.
EFI_NOT_READY	The deauthentication service is invoked to a nonexistent association or authentication relationship.
EFI_NOT_FOUND	The specified peer MAC entity is not found.

continues on next page

Table 27.17 – continued from previous page

EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.
----------------------	---

27.4 EFI Wireless MAC Connection II Protocol

This section provides a detailed description of EFI Wireless MAC Connection II Protocol.

27.4.1 EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL

Summary

The *EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL* provides network management service interfaces for 802.11 network stack. It is used by network applications (and drivers) to establish wireless connection with a wireless network.

GUID

```
#define EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL_GUID \
{ 0x1b0fb9bf, 0x699d, 0x4fdd, \
  { 0xa7, 0xc3, 0x25, 0x46, 0x68, 0x1b, 0xf6, 0x3b } }
```

Protocol Interface Structure

```
typedef struct _EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL {
    EFI_WIRELESS_MAC_CONNECTION_II_GET_NETWORKS      GetNetworks;
    EFI_WIRELESS_MAC_CONNECTION_II_CONNECT_NETWORK    ConnectNetwork;
    EFI_WIRELESS_MAC_CONNECTION_II_DISCONNECT_NETWORK DisconnectNetwork;
} EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL;
```

Parameters

GetNetworks

Get a list of nearby detectable wireless network. See the *GetNetworks()* function description.

ConnectNetwork

Places a connection request with a specific wireless network. See the *ConnectNetwork()* function description.

DisconnectNetwork

Places a disconnection request with a specific wireless network. See the *DisconnectNetwork()* function description.

Description

The *EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL* is designed to provide management service interfaces for the EFI wireless network stack to establish relationship with a wireless network (identified by *EFI_80211_NETWORK* defined below). An EFI Wireless MAC Connection II Protocol instance will be installed on each communication device that the EFI wireless network stack runs on.

27.4.2 EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL.GetNetworks()

Summary

Request a survey of potential wireless networks that administrator can later elect to try to join.

Prototype

```
typedef
EFI_STATUS
(EFI_API *EFI_WIRELESS_MAC_CONNECTION_II_GET_NETWORKS)(
    IN EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL      *This,
    IN EFI_80211_GET_NETWORKS_TOKEN                 *Token
);
```

Parameters

This

Pointer to the *EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL* instance.

Token

Pointer to the token for getting wireless network. Type *EFI_80211_GET_NETWORKS_TOKEN* is defined in Related Definitions below.

Description

The *GetNetworks()* function returns the description of a list of wireless networks detected by wireless UEFI driver. This function is always non-blocking. If the operation succeeds or fails due to any error, the *Token->Event* will be signaled and *Token->Status* will be updated accordingly. The caller of this function is responsible for inputting SSIDs in case of searching hidden networks.

Related Definitions

```
/**
//*****
// EFI_80211_GET_NETWORKS_TOKEN
//*****
typedef struct {
    EFI_EVENT          Event;
    EFI_STATUS         Status;
    EFI_80211_GET_NETWORKS_DATA *Data;
    EFI_80211_GET_NETWORKS_RESULT *Result;
} EFI_80211_GET_NETWORKS_TOKEN;
```

Event

If the status code returned by *GetNetworks()* is *EFI_SUCCESS*, then this Event will be signaled after the Status field is updated by the EFI Wireless MAC Connection Protocol II driver. The type of Event must be *EFI_NOTIFY_SIGNAL*.

Status

Will be set to one of the following values:

EFI_SUCCESS: The operation completed successfully.

EFI_NOT_FOUND: Failed to find available wireless networks.

EFI_DEVICE_ERROR: An unexpected network or system error occurred.

EFI_ACCESS_DENIED: The operation is not completed due to some underlying hardware or software state.

EFI_NOT_READY: The operation is started but not yet completed.

Data

Pointer to the input data for getting networks. Type `EFI_80211_GET_NETWORKS_DATA` is defined below.

Result

Indicates the scan result. It is caller's responsibility to free this buffer. Type `EFI_80211_GET_NETWORKS_RESULT` is defined below.

```
//*****
// EFI_80211_GET_NETWORKS_DATA
//*****
typedef struct {
    UINT32          NumOfSSID;
    EFI_80211_SSID  SSIDList[1];
} EFI_80211_GET_NETWORKS_DATA;
```

NumOfSSID

The number of `EFI_80211_SSID` in `SSIDList`. If zero, `SSIDList` should be ignored.

SSIDList

The `SSIDList` is a pointer to an array of `EFI_80211_SSID` instances. The number of entries is specified by `NumOfSSID`. The array should only include SSIDs of hidden networks. It is suggested that the caller inputs less than 10 elements in the `SSIDList`. It is the caller's responsibility to free this buffer. Type `EFI_80211_SSID` is defined below.

```
#define EFI_MAX_SSID_LEN 32

//*****
// EFI_80211_SSID
//*****
typedef struct {
    UINT8          SSIdLen;
    UINT8          SSId[EFI_MAX_SSID_LEN];
} EFI_80211_SSID;
```

SSIdLen

Length in bytes of the SSId. If zero, ignore SSId field.

SSId

Specifies the service set identifier.

```
//*****
// EFI_80211_GET_NETWORKS_RESULT
//*****
typedef struct {
    UINT8          NumOfNetworkDesc;
    EFI_80211_NETWORK_DESCRIPTION  NetworkDesc[1];
} EFI_80211_GET_NETWORKS_RESULT;
```

NumOfNetworkDesc

The number of elements in `NetworkDesc`. If zero, `NetworkDesc` should be ignored.

NetworkDesc

The `NetworkDesc` is a variable-length array of elements of type `EFI_80211_NETWORK_DESCRIPTION`. Type `EFI_80211_NETWORK_DESCRIPTION` is defined below.

```

//*****
// EFI_80211_NETWORK_DESCRIPTION
//*****
typedef struct {
    EFI_80211_NETWORK      Network;
    UINT8                  NetworkQuality;
} EFI_80211_NETWORK_DESCRIPTION;

```

Network

Specifies the found wireless network. Type EFI_80211_NETWORK is defined below.

NetworkQuality

Indicates the network quality as a value between 0 to 100, where 100 indicates the highest network quality.

```

//*****
// EFI_80211_NETWORK
//*****
typedef struct {
    EFI_80211_BSS_TYPE      BSSType;
    EFI_80211_SSID          SSId;
    EFI_80211_AKM_SUITE_SELECTOR *AKMSuite;
    EFI_80211_CIPHER_SUITE_SELECTOR *CipherSuite;
} EFI_80211_NETWORK;

```

BSSType

Specifies the type of the BSS. Type EFI_80211_BSS_TYPE is defined below.

SSId

Specifies the SSID of the BSS. Type EFI_80211_SSID is defined above.

AKMSuite

Pointer to the AKM suites supported in the wireless network. Type EFI_80211_AKM_SUITE_SELECTOR is defined below.

CipherSuite

Pointer to the cipher suites supported in the wireless network. Type EFI_80211_CIPHER_SUITE_SELECTOR is defined below.

```

//*****
// EFI_80211_BSS_TYPE
//*****
typedef enum {
    IeeeInfrastructureBSS,
    IeeeIndependentBSS,
    IeeeMeshBSS,
    IeeeAnyBss
} EFI_80211_BSS_TYPE;

```

The *EFI_80211_BSS_TYPE* is defined to enumerate BSS type.

```

//*****
// EFI_80211_SUITE_SELECTOR
//*****
typedef struct {
    UINT8      Oui[3];

```

(continues on next page)

(continued from previous page)

```
UINT8      SuiteType;
} EFI_80211_SUITE_SELECTOR;
```

Oui

Organization Unique Identifier, as defined in IEEE 802.11 standard, usually set to 00-0F-AC.

SuiteType

Suites types, as defined in IEEE 802.11 standard.

```
/**
 * *****
 */
// EFI_80211_AKM_SUITE_SELECTOR
// *****
typedef struct {
    UINT16      AKMSuiteCount;
    EFI_80211_SUITE_SELECTOR AKMSuiteList[1];
} EFI_80211_AKM_SUITE_SELECTOR;
```

AKMSuiteCount

Indicates the number of AKM suite selectors that are contained in AKMSuiteList. If zero, the AKMSuiteList is ignored.

AKMSuiteList

A variable-length array of AKM suites, as defined in IEEE 802.11 standard, Table 8-101. The number of entries is specified by AKMSuiteCount.

```
/**
 * *****
 */
// EFI_80211_CIPHER_SUITE_SELECTOR
// *****
typedef struct {
    UINT16      CipherSuiteCount;
    EFI_80211_SUITE_SELECTOR CipherSuiteList[1];
} EFI_80211_CIPHER_SUITE_SELECTOR;
```

CipherSuiteCount

Indicates the number of cipher suites that are contained in CipherSuiteList. If zero, the CipherSuiteList is ignored.

CipherSuiteList

A variable-length array of cipher suites, as defined in IEEE 802.11 standard, Table 8-99. The number of entries is specified by CipherSuiteCount.

Status Codes Returned

EFI_SUCCESS	The operation started, and an event will eventually be raised for the caller.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: This is NULL. Token is NULL.
EFI_UNSUPPORTED	One or more of the input parameters is not supported by this implementation.
EFI_ALREADY_STARTED	The operation of getting wireless network is already started.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

27.4.3 EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL.ConnectNetwork()

Summary

Connect a wireless network specified by a particular SSID, BSS type and Security type.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_WIRELESS_MAC_CONNECTION_II_CONNECT_NETWORK) (
    IN EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL      *This,
    IN EFI_80211_CONNECT_NETWORK_TOKEN              *Token
);
```

Parameters

This

Pointer to the *EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL* instance.

Token

Pointer to the token for connecting wireless network. Type *EFI_80211_CONNECT_NETWORK_TOKEN* is defined in Related Definitions below.

Description

The *ConnectNetwork()* function places a request to wireless UNDI driver to connect a wireless network specified by a particular SSID, BSS type, Authentication method and cipher. This function will trigger wireless UNDI driver to perform authentication and association process to establish connection with a particular Access Point for the specified network. This function is always non-blocking. If the connection succeeds or fails due to any error, the *Token->Event* will be signaled and *Token->Status* will be updated accordingly.

After having signaled a successful connection completion, the UNDI driver will update the network connection state using the network media state information type in the *EFI_ADAPTER_INFORMATION_PROTOCOL*. If needed, the caller should use *EFI_ADAPTER_INFORMATION_PROTOCOL* to regularly get the network media state to find if the UNDI driver is still connected to the wireless network (*EFI_SUCCESS*) or not (*EFI_NO_MEDIA*).

Generally a driver or application in WiFi stack would provide user interface to end user to manage profiles for selecting which wireless network to join and other state management. This module should prompt the user to select a network and input WiFi security data such as certificate, private key file, password, etc. Then the module should deploy WiFi security data through EFI Supplicant Protocol and/or EFI EAP Configuration Protocol before calling *ConnectNetwork()* function.

Related Definitions

```
/**
//*****
//  EFI_80211_CONNECT_NETWORK_TOKEN
//*****
typedef struct {
    EFI_EVENT                      Event;
    EFI_STATUS                    Status;
    EFI_80211_CONNECT_NETWORK_DATA *Data;
    EFI_80211_CONNECT_NETWORK_RESULT_CODE ResultCode;
} EFI_80211_CONNECT_NETWORK_TOKEN;
```

Event

If the status code returned by `ConnectNetwork()` is `EFI_SUCCESS`, then this Event will be signaled after the Status field is updated by the EFI Wireless MAC Connection Protocol II driver. The type of Event must be `EFI_NOTIFY_SIGNAL`.

Status

Will be set to one of the following values:

`EFI_SUCCESS`: The operation completed successfully.

`EFI_DEVICE_ERROR`: An unexpected network or system error occurred.

`EFI_ACCESS_DENIED`: The operation is not completed due to some underlying hardware or software state.

`EFI_NOT_READY`: The operation is started but not yet completed.

Data

Pointer to the connection data. Type `EFI_80211_CONNECT_NETWORK_DATA` is defined below.

ResultCode

Indicates the connection state. Type `EFI_80211_CONNECT_NETWORK_RESULT_CODE` is defined below.

The `EFI_80211_CONNECT_NETWORK_TOKEN` structure is defined to support the process of determining the characteristics of the available networks. As input, the *Data* field must be filled in by the caller of EFI Wireless MAC Connection II Protocol. After the operation completes, the EFI Wireless MAC Connection II Protocol driver updates the *Status* and *ResultCode* field and the *Event* is signaled.

```

//*****
// EFI_80211_CONNECT_NETWORK_DATA
//*****
typedef struct {
    EFI_80211_NETWORK      *Network;
    UINT32                  FailureTimeout;
} EFI_80211_CONNECT_NETWORK_DATA;

```

Network

Specifies the wireless network to connect to. Type `EFI_80211_NETWORK` is defined above.

FailureTimeout

Specifies a time limit in seconds that is optionally present, after which the connection establishment procedure is terminated by the UNDI driver. This is an optional parameter and may be 0. Values of 5 seconds or higher are recommended.

```

//*****
// EFI_80211_CONNECT_NETWORK_RESULT_CODE
//*****
typedef enum {
    ConnectSuccess,
    ConnectRefused,
    ConnectFailed,
    ConnectFailureTimeout,
    ConnectFailedReasonUnspecified
} EFI_80211_CONNECT_NETWORK_RESULT_CODE;

```

ConnectSuccess

The connection establishment operation finished successfully.

ConnectRefused

The connection was refused by the Network.

ConnectFailed

The connection establishment operation failed (i.e, Network is not detected).

ConnectFailureTimeout

The connection establishment operation was terminated on timeout.

ConnectFailedReasonUnspecified

The connection establishment operation failed on other reason.

Status Codes Returned

EFI_SUCCESS	The operation started successfully. Results will be notified eventually.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: This is NULL. Token is NULL.
EFI_UNSUPPORTED	One or more of the input parameters are not supported by this implementation.
EFI_ALREADY_STARTED	The connection process is already started.
EFI_NOT_FOUND	The specified wireless network is not found.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

27.4.4 EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL.DisconnectNetwork()

Summary

Request a disconnection **with** current connected wireless network.

Prototype

typedef

EFI_STATUS

(EFI_API *EFI_WIRELESS_MAC_CONNECTION_II_DISCONNECT_NETWORK) (

IN EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL *This,

IN EFI_80211_DISCONNECT_NETWORK_TOKEN *Token

);

Parameters

This

Pointer to the *EFI_WIRELESS_MAC_CONNECTION_II_PROTOCOL* instance.

Token

Pointer to the token for disconnecting wireless network. Type *EFI_80211_DISCONNECT_NETWORK_TOKEN* is defined in Related Definitions below.

Description

The *DisconnectNetwork()* function places a request to wireless UEFI driver to disconnect from the wireless network it is connected to. This function will trigger the wireless UEFI driver to perform disassociation and deauthentication process to terminate an existing connection. This function is always non-blocking. After wireless UEFI driver received

acknowledgment frame from AP and freed up corresponding resources, the *Token->Event* will be signaled and *Token->Status* will be updated accordingly.

Related Definitions

```

//*****
// EFI_80211_DISCONNECT_NETWORK_TOKEN
//*****
typedef struct {
    EFI_EVENT          Event;
    EFI_STATUS         Status;
} EFI_80211_DISCONNECT_NETWORK_TOKEN;

```

Event

If the status code returned by `DisconnectNetwork()` is `EFI_SUCCESS`, then this Event will be signaled after the Status field is updated by the EFI Wireless MAC Connection Protocol II driver. The type of Event must be `EFI_NOTIFY_SIGNAL`.

Status

Will be set to one of the following values:

`EFI_SUCCESS`: The operation completed successfully

`EFI_DEVICE_ERROR`: An unexpected network or system error occurred.

`EFI_ACCESS_DENIED`: The operation is not completed due to some underlying hardware or software state.

Status Codes Returned

<code>EFI_SUCCESS</code>	The operation started successfully. Results will be notified eventually.
<code>EFI_INVALID_PARAMETER</code>	One or more of the following conditions is TRUE: This is NULL. Token is NULL.
<code>EFI_UNSUPPORTED</code>	One or more of the input parameters are not supported by this implementation.
<code>EFI_NOT_FOUND</code>	Not connected to a wireless network.
<code>EFI_OUT_OF_RESOURCES</code>	Required system resources could not be allocated.

27.5 EFI Supplicant Protocol

This section defines the EFI Supplicant Protocol.

27.5.1 Suppliant Service Binding Protocol

27.5.2 EFI_SUPPLICANT_SERVICE_BINDING_PROTOCOL

Summary

The EFI Suppliant Service Binding Protocol is used to locate EFI Suppliant Protocol drivers to create and destroy child of the driver to communicate with other host using Suppliant protocol.

GUID

```
#define EFI_SUPPLICANT_SERVICE_BINDING_PROTOCOL_GUID \
{ 0x45bcd98e, 0x59ad, 0x4174, \
  { 0x95, 0x46, 0x34, 0x4a, 0x7, 0x48, 0x58, 0x98 } }
```

Description

A module that requires suppliant services can call one of the protocol handler services, such as *BS->LocateHandleBuffer()*, to search devices that publish an EFI Suppliant Service Binding Protocol GUID. Such device supports the EFI Suppliant Protocol and may be available for use. After a successful call to the *EFI_SUPPLICANT_SERVICE_BINDING_PROTOCOL.CreateChild()* function, the newly created child EFI Suppliant Protocol driver is in an un-configured state; it is not ready to do any operation until configured via *SetData()*. Every successful call to the *EFI_SUPPLICANT_SERVICE_BINDING_PROTOCOL.CreateChild()* function must be matched with a call to the *EFI_SUPPLICANT_SERVICE_BINDING_PROTOCOL.DestroyChild()* function to release the protocol driver.

27.5.3 Suppliant Protocol

27.5.4 EFI_SUPPLICANT_PROTOCOL

Summary

This protocol provides services to process authentication and data encryption/decryption for security management.

GUID

```
#define EFI_SUPPLICANT_PROTOCOL_GUID \
{ 0x54fcc43e, 0xaa89, 0x4333, \
  { 0x9a, 0x85, 0xcd, 0xea, 0x24, 0x5, 0x1e, 0x9e } }
```

Protocol Interface Structure

```
typedef struct _EFI_SUPPLICANT_PROTOCOL {
    EFI_SUPPLICANT_BUILD_RESPONSE_PACKET    BuildResponsePacket;
    EFI_SUPPLICANT_PROCESS_PACKET           ProcessPacket;
    EFI_SUPPLICANT_SET_DATA                 SetData;
    EFI_SUPPLICANT_GET_DATA                 GetData;
} EFI_SUPPLICANT_PROTOCOL;
```

Parameters

BuildResponsePacket

This API processes security data for handling key management. See the *BuildResponsePacket()* function description.

ProcessPacket

This API processes frame for encryption or decryption. See the *ProcessPacket()* function description.

SetData

This API sets the information needed during key generated in handshake. See the *SetData()* function description.

GetData

This API gets the information generated in handshake. See the *GetData()* function description.

Description

The *EFI_SUPPLICANT_PROTOCOL* is designed to provide unified place for WIFI and EAP security management. Both PSK authentication and 802.1X EAP authentication can be managed via this protocol and driver or application as a consumer can only focus on about packet transmitting or receiving. For 802.1X EAP authentication, an instance of *EFI_EAP_CONFIGURATION_PROTOCOL* must be installed to the same handle as the EFI Supplicant Protocol.

27.5.5 EFI_SUPPLICANT_PROTOCOL.BuildResponsePacket()

Summary

BuildResponsePacket() is called during STA and AP authentication is in progress. Supplicant derives the PTK or session keys depend on type of authentication is being employed.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_SUPPLICANT_BUILD_RESPONSE_PACKET) (
    IN EFI_SUPPLICANT_PROTOCOL          *This,
    IN UINT8                            *RequestBuffer, OPTIONAL
    IN UINTN                            RequestBufferSize, OPTIONAL
    OUT UINT8                           *Buffer,
    IN OUT UINTN                        *BufferSize
);
```

Parameters

This

Pointer to the *EFI_SUPPLICANT_PROTOCOL* instance.

RequestBuffer

Pointer to the most recently received EAPOL packet. *NULL* means the supplicant need initiate the EAP authentication session and send EAPOL-Start message.

RequestSize

Packet size in bytes for the most recently received EAPOL packet. 0 is only valid when *RequestBuffer* is *NULL*.

Buffer

Pointer to the buffer to hold the built packet.

BufferSize

Pointer to the buffer size in bytes. On input, it is the buffer size provided by the caller. On output, it is the buffer size in fact needed to contain the packet.

Description

The consumer calls *BuildResponsePacket()* when it receives the security frame. It simply passes the data to supplicant to process the data. It could be WPA-PSK which starts the 4-way handshake, or WPA-EAP first starts with Authentication process and then 4-way handshake, or 2-way group key handshake. In process of authentication, 4-way handshake or group key handshake, Supplicant needs to communicate with its peer (AP/AS) to fill the output buffer parameter. Once the 4 way handshake or group key handshake is over, and PTK (Pairwise Transient keys) and GTK (Group Temporal Key) are generated.

Status Codes Returned

EFI_SUCCESS	The required EAPOL packet is built successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>RequestBuffer</i> is <i>NULL</i>, but <i>RequestSize</i> is NOT 0. • <i>RequestSize</i> is 0. • <i>Buffer</i> is <i>NULL</i>, but <i>RequestBuffer</i> is NOT 0. • <i>RequestSize</i> is 0. • <i>BufferSize</i> is <i>NULL</i>.
EFI_BUFFER_TOO_SMALL	<i>BufferSize</i> is too small to hold the response packet.
EFI_NOT_READY	Current EAPOL session state is NOT ready to build <i>ResponsePacket</i> .

27.5.6 EFI_SUPPLICANT_PROTOCOL.ProcessPacket()

Summary

ProcessPacket() is called to Supplicant driver to encrypt or decrypt the data depending type of authentication type.

Prototype

```
typedef
EFI_STATUS
(EFI_API *EFI_SUPPLICANT_PROCESS_PACKET)(
    IN EFI_SUPPLICANT_PROTOCOL          *This,
    IN OUT EFI_SUPPLICANT_FRAGMENT_DATA **FragmentTable,
    IN UINT32                           *FragmentCount,
    IN EFI_SUPPLICANT_CRYPT_MODE        CryptMode
);
```

Parameters

This

Pointer to the *EFI_SUPPLICANT_PROTOCOL* instance.

FragmentTable

Pointer to a list of fragment. The caller will take responsible to handle the original *FragmentTable* while it may be reallocated in Supplicant driver.

FragmentCount

Number of fragment.

CryptMode

Crypt mode.

Description

ProcessPacket() is responsible for encrypting or decrypting the data traffic as per authentication type. The consumer routes the data frame as it is to Supplicant module and encrypts or decrypts packet with updated length comes as output parameter. Supplicant holds the derived PTK and GTKs and uses this key to encrypt or decrypt the network traffic.

If the Supplicant driver does not support any encryption and decryption algorithm, then *EFI_UNSUPPORTED* is returned.

Related Definitions

```

//*****
// EFI_SUPPLICANT_FRAGMENT_DATA
//*****
typedef struct {
    UINT32          FragmentLength;
    VOID            *FragmentBuffer;
} EFI_SUPPLICANT_FRAGMENT_DATA;

```

FragmentLength

Length of data buffer in the fragment.

FragmentBuffer

Pointer to the data buffer in the fragment.

```

//*****
// EFI_SUPPLICANT_CRYPT_MODE
//*****
typedef enum {
    EfiSupplicantEncrypt,
    EfiSupplicantDecrypt,
} EFI_SUPPLICANT_CRYPT_MODE;

```

EfiSupplicantEncrypt

Encrypt data provided in the fragment buffers.

EfiSupplicantDecrypt

Decrypt data provided in the fragment buffers.

Status Codes Returned

EFI_SUCCESS	The operation completed successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>FragmentTable</i> is NULL. • <i>FragmentCount</i> is NULL. • <i>CryptMode</i> is invalid.
EFI_NOT_READY	Current supplicant state is NOT Authenticated.
EFI_ABORTED	Something wrong decryption the message.
EFI_UNSUPPORTED	This API is not supported.

27.5.7 EFI_SUPPLICANT_PROTOCOL.SetData()

Summary

Set Supplicant configuration data.

Prototype

```

typedef
EFI_STATUS
(EFIAPI *EFI_SUPPLICANT_SET_DATA)(
    IN EFI_SUPPLICANT_PROTOCOL      *This,

```

(continues on next page)

(continued from previous page)

```

IN EFI_SUPPLICANT_DATA_TYPE      DataType,
IN VOID                          *Data,
IN UINTN                        DataSize
);

```

Parameters**This**

Pointer to the *EFI_SUPPLICANT_PROTOCOL* instance.

DataType

The type of data.

Data

Pointer to the buffer to hold the data.

DataSize

Pointer to the buffer size in bytes.

Description

The *SetData()* function sets Supplicant configuration. For example, Supplicant driver need to know Password and TargetSSIDName to calculate PSK. Supplicant driver need to know StationMac and TargetSSIDMac to calculate PTK. Then it can derive KCK(key confirmation key) which is needed to calculate MIC, and KEK(key encryption key) which is needed to unwrap GTK.

Related Definitions

```

//*****
//  EFI_SUPPLICANT_DATA_TYPE
//*****
typedef enum {
    //
    // Session Configuration
    //
    EfiSupplicant80211AKMSuite,
    EfiSupplicant80211GroupDataCipherSuite,
    EfiSupplicant80211PairwiseCipherSuite,
    EfiSupplicant80211PskPassword,
    EfiSupplicant80211TargetSSIDName,
    EfiSupplicant80211StationMac,
    EfiSupplicant80211TargetSSIDMac,
    //
    // Session Information
    //
    EfiSupplicant80211PTK,
    EfiSupplicant80211GTK,
    EfiSupplicantState,
    EfiSupplicant80211LinkState,
    EfiSupplicantKeyRefresh,

    //
    // Session Configuration
    //
    EfiSupplicant80211SupportedAKMSuites,
    EfiSupplicant80211SupportedSoftwareCipherSuites,

```

(continues on next page)

(continued from previous page)

```
EfiSupplicant80211SupportedHardwareCipherSuites,

//
// Session Information
//
EfiSupplicant80211IGTK,
EfiSupplicant80211PMK,
EfiSupplicantDataTypeMaximum
}   EFI_SUPPLICANT_DATA_TYPE;
```

EfiSupplicant80211AKMSuite

Current authentication type in use. The corresponding Data is of type EFI_80211_AKM_SUITE_SELECTOR.

EfiSupplicant80211GroupDataCipherSuite

Group data encryption type in use. The corresponding Data is of type EFI_80211_CIPHER_SUITE_SELECTOR.

EfiSupplicant80211PairwiseCipherSuite

Pairwise encryption type in use. The corresponding Data is of type EFI_80211_CIPHER_SUITE_SELECTOR.

EfiSupplicant80211PskPassword

PSK password. The corresponding Data is a NULL-terminated ASCII string.

EfiSupplicant80211TargetSSIDName

Target SSID name. The corresponding Data is of type EFI_80211_SSID.

EfiSupplicant80211StationMac

Station MAC address. The corresponding Data is of type EFI_80211_MAC_ADDRESS.

EfiSupplicant80211TargetSSIDMac

Target SSID MAC address. The corresponding Data is 6 bytes MAC address.

EfiSupplicant80211PTK

802.11 PTK. The corresponding Data is of type EFI_SUPPLICANT_KEY.

EfiSupplicant80211GTK

802.11 GTK. The corresponding Data is of type EFI_SUPPLICANT_GTK_LIST.

EfiSupplicantState

Supplicant state. The corresponding Data is EFI_EAPOL_SUPPLICANT_PAE_STATE.

EfiSupplicant80211LinkState

802.11 link state. The corresponding Data is EFI_80211_LINK_STATE.

EfiSupplicantKeyRefresh

Flag indicates key is refreshed. The corresponding Data is EFI_SUPPLICANT_KEY_REFRESH.

EfiSupplicant80211SupportedAKMSuites

Supported authentication types. The corresponding Data is of type EFI_80211_AKM_SUITE_SELECTOR.

EfiSupplicant80211SupportedSoftwareCipherSuites

Supported software encryption types provided by supplicant driver. The corresponding Data is of type EFI_80211_CIPHER_SUITE_SELECTOR.

EfiSupplicant80211SupportedHardwareCipherSuites

Supported hardware encryption types provided by wireless UNDI driver. The corresponding Data is of type EFI_80211_CIPHER_SUITE_SELECTOR.

EfiSupplicant80211GTK

802.11 Integrity GTK. The corresponding Data is of type EFI_SUPPLICANT_GTK_LIST.

EfiSupplicant80211IPMK

802.11 PMK. The corresponding Data is 32 bytes pairwise master key.

```
//*****
// EFI_80211_LINK_STATE
//*****
typedef enum {
    Ieee80211UnauthenticatedUnassociated,
    Ieee80211AuthenticatedUnassociated,
    Ieee80211PendingRSNAAuthentication,
    Ieee80211AuthenticatedAssociated
} EFI_80211_LINK_STATE;
```

Ieee80211UnauthenticatedUnassociated

Indicates initial start state, unauthenticated, unassociated.

Ieee80211AuthenticatedUnassociated

Indicates authenticated, unassociated.

Ieee80211PendingRSNAAuthentication

Indicates authenticated and associated, but pending RSN authentication.

Ieee80211AuthenticatedAssociated

Indicates authenticated and associated.

```
//*****
// EFI_SUPPLICANT_KEY_REFRESH
//*****
typedef struct {
    BOOLEAN                GTKRefresh;
} EFI_SUPPLICANT_KEY_REFRESH;
```

GTKRefresh

If TRUE, indicates GTK is just refreshed after a successful call to EFI_SUPPLICANT_PROTOCOL.BuildResponsePacket().

```
//*****
// EFI_SUPPLICANT_GTK_LIST
//*****
typedef struct {
    UINT8                GTKCount;
    EFI_SUPPLICANT_KEY    GTKList[1];
} EFI_SUPPLICANT_GTK_LIST;
```

GTKCount

Indicates the number of GTKs that are contained in GTKList.

GTKList

A variable-length array of GTKs of type EFI_SUPPLICANT_KEY. The number of entries is specified by GTKCount.

```
#define EFI_MAX_KEY_LEN 64
```

(continues on next page)

(continued from previous page)

```

//*****
// EFI_SUPPLICANT_KEY
//*****
typedef struct {
    UINT8                Key[EFI_MAX_KEY_LEN];
    UINT8                KeyLen;
    UINT8                KeyId;
    EFI_SUPPLICANT_KEY_TYPE KeyType;
    EFI_80211_MAC_ADDRESS Addr;
    UINT8                Rsc[8];
    UINT8                RscLen;
    BOOLEAN              IsAuthenticator;
    EFI_80211_SUITE_SELECTOR CipherSuite;
    EFI_SUPPLICANT_KEY_DIRECTION Direction;
} EFI_SUPPLICANT_KEY;

```

The EFI_SUPPLICANT_KEY descriptor is defined in the IEEE 802.11 standard, section 6.3.19.1.2.

Key

The key value.

KeyLen

Length in bytes of the Key. Should be up to EFI_MAX_KEY_LEN.

KeyId

The key identifier.

KeyType

Defines whether this key is a group key, pairwise key, PeerKey, or Integrity Group.

Addr

The value is set according to the KeyType.

RSC

The Receive Sequence Count value.

RscLen

Length in bytes of the Rsc. Should be up to 8.

IsAuthenticator

Indicates whether the key is configured by the Authenticator or Supplicant. The value true indicates Authenticator.

CipherSuite

The cipher suite required for this association.

Direction

Indicates the direction for which the keys are to be installed.

```

//*****
// EFI_SUPPLICANT_KEY_TYPE (IEEE Std 802.11
//      Section 6.3.19.1.2)
//*****
typedef enum {
    Group,
    Pairwise,
    PeerKey,

```

(continues on next page)

(continued from previous page)

```
IGTK
}   EFI_SUPPLICANT_KEY_TYPE;
```

The EFI_SUPPLICANT_KEY_TYPE is defined in the IEEE 802.11 specification.

```
/**
// *****
// EFI_SUPPLICANT_KEY_DIRECTION (IEEE Std 802.11
//     Section 6.3.19.1.2)
// *****
typedef enum {
    Receive,
    Transmit,
    Both
}   EFI_SUPPLICANT_KEY_DIRECTION;
```

Receive

Indicates that the keys are being installed for the receive direction.

Transmit

Indicates that the keys are being installed for the transmit direction.

Both

Indicates that the keys are being installed for both the receive and transmit directions.

Status Codes Returned

EFI_SUCCESS	The Supplicant configuration data is set successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • <i>Data</i> is NULL. • <i>DataSize</i> is 0.
EFI_UNSUPPORTED	The <i>DataType</i> is unsupported.
EFI_OUT_OF_RESOURCES	Required system resources could not be allocated.

27.5.8 EFI_SUPPLICANT_PROTOCOL.GetData()

Summary

Get Supplicant configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_SUPPLICANT_GET_DATA)(
    IN EFI_SUPPLICANT_PROTOCOL      *This,
    IN EFI_SUPPLICANT_DATA_TYPE     DataType,
    OUT UINT8                       *Data, OPTIONAL
    IN OUT UINTN                    *DataSize
);
```

Parameters

This

Pointer to the *EFI_SUPPLICANT_PROTOCOL* instance.

DataType

The type of data.

Data

Pointer to the buffer to hold the data. Ignored if *DataSize* is 0.

DataSize

Pointer to the buffer size in bytes. On input, it is the buffer size provided by the caller. On output, it is the buffer size in fact needed to contain the packet.

Description

The *GetData()* function gets Supplicant configuration. The typical example is PTK and GTK derived from handshake. The wireless NIC can support software encryption or hardware encryption. If the consumer uses software encryption, it can call *ProcessPacket()* to get result. If the consumer supports hardware encryption, it can get PTK and GTK via *GetData()* and program to hardware register.

Status Codes Returned

EFI_SUCCESS	The Supplicant configuration data is got successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • This is NULL. • DataSize is NULL. • Data is NULL if DataSize is not zero.
EFI_UNSUPPORTED	The <i>DataType</i> is unsupported.
EFI_NOT_FOUND	The Supplicant configuration data is not found.
EFI_BUFFER_TOO_SMALL	The size of <i>Data</i> is too small for the specified configuration data and the required size is returned in <i>DataSize</i> .